Parallel Oblivious Acyclic Joins for TEE-based Encrypted Databases

Sen Zhao[†], Binbin Gu[‡], Haibo Hu[¶], Xiaofeng Chen[†*]
[†]Xidian University [‡]Nanyang Technological University [¶]The Hong Kong Polytechnic University
senzhao@stu.xidian.edu.cn, binbin.gu@ntu.edu.sg, haibo.hu@polyu.edu.hk, xfchen@xidian.edu.cn

***Abstract*—Outsourced databases increasingly rely on Trusted Execution Environments (TEEs) to process sensitive data inside a hardware-protected enclave, but data-dependent memory access patterns can still leak private information. Oblivious algorithms close this channel and are well understood for binary joins. For acyclic multi-way joins, however, state-of-the-art frameworks still inherit the Yannakakis structure, whose final phase materializes the result through a chain of oblivious binary joins: each join consumes the intermediate of the previous one and re-materializes it at the output size, so the steps run serially across sibling subtrees and repeatedly invoke the most expensive oblivious primitive. This chain remains the bottleneck no matter how well each operator is parallelized.**

We propose JFYan, a parallel oblivious *join-free* Yannakakis framework that eliminates this chain rather than parallelizing it. The key idea is to separate *counting* the output from *writing* it: a bottom-up pass computes how many output rows each tuple contributes, a single expansion at the root fixes all output positions, and a top-down pass hands these positions to descendant subtrees, which then proceed in parallel. The whole algorithm performs exactly one output-size expansion, its dependency span is governed only by the join-tree depth, and a single public materialization target lets it realize full, advised-full, and differential obliviousness. On TPC-DS and TPC-H workloads, JFYan consistently outperforms both the state-of-the-art scheme and our operator-parallelized variant of it. On Query 85 at scale factor 30, it runs in 38.13 seconds, achieving a $22.44\times$ speedup over the state-of-the-art scheme and still clearly outperforming the operator-parallelized variant.

***Index Terms*—Encrypted Databases, Trusted Execution Environments, Oblivious Joins**

I. Introduction

Outsourcing databases to cloud servers allows resource-constrained clients to store and query data without maintaining local infrastructure [1], [2], but it also places the data outside the owner's direct control, raising concerns about the exposure of sensitive information [3]. A common remedy is to encrypt the database before outsourcing; however, crypto-based databases remain impractical for general relational query processing due to their prohibitive overhead [4]–[7]. TEEs provide a practical alternative: data stays encrypted outside a hardware-protected enclave while queries are processed in plaintext inside it, supporting general relational queries efficiently in practice [8]–[11].

Although TEEs protect data and computation inside the enclave, query execution may still reveal data-dependent access patterns to the untrusted server [12]–[15]. Consider an acyclic join $R_1(A,B) \bowtie R_2(B,C) \bowtie R_3(C,D)$. Two neighboring instances can have the same relation sizes and the same output size $\tau = 2$, yet induce different traces: in one, the two outputs pass through distinct tuples of R_2 and R_3; in the other, after one tuple of R_1 is modified so that both join on the same key, both outputs traverse the same tuple of R_2 and the same tuple of R_3. The two evaluations thus touch memory in distinguishable ways, so the untrusted server can tell the instances apart. Moreover, the concentration of accesses reveals structural information such as key skew, tuple participation, and join-key degrees, which may be combined with auxiliary knowledge to infer sensitive information [16]–[18].

Achieving *full obliviousness (FO)* [12], [13], [19], [20], which requires access patterns to be independent of the underlying data except for public sizes such as the exact output size τ, has become a key requirement for TEE-based encrypted databases. Recent work targets *output-size-hiding obliviousness* variants in multi-way joins: *advised full obliviousness (a-FO)* pads the computation to a public and data-independent worst-case bound Λ_{aFO} (the AGM bound N^{ρ}, where ρ is the fractional edge-cover number), while *differential obliviousness (DO)* pads it to a differentially private upper bound Λ_{DO} [18]. These models specify the security target; they do not specify how the join should be executed efficiently.

Realizing any of these targets in practice requires an efficient oblivious execution framework, and so far such frameworks have been developed only under the *FO* model. Oblivious binary joins are well studied [13], [21], [22], and the first *parallel* oblivious binary join has recently appeared [15]. For multi-way joins, however, no *parallel* oblivious solution exists, even for acyclic joins [23]–[27], an important and widely studied subclass [17], [28]. The state-of-the-art scheme is ObliYan [17], an oblivious instantiation of the classical three-phase Yannakakis algorithm [23], [29]: two semi-join phases (Phases I and II) strip away dangling tuples, and a final phase (Phase III) materializes the result bottom-up along the join tree, in $O(N+\tau)$ time overall. Making every operator oblivious inflates the cost to $O(m(\tau+N)\log^2(\tau+N))$ over m relations, and two obstacles separate this oblivious cost from a *parallel* one. The first is at the *operator* level: the oblivious operators invoked across all phases are each internally sequential. This obstacle is the more tractable one, since every operator admits a parallel substitute: the recent parallel binary join POTWJ [15] covers the materialization phase, and a parallel

* Corresponding author.

TABLE I: Complexity comparison under a common public target Λ (Section V-C), over m relations of total size N, with join-tree depth d, maximum branching factor k, and materialization depth $D(\mathcal{T})$ (Definition II.2); $L = \log^2(\Lambda + N)$. W: total work; S: span (longest dependency path); p: the number of cores.

Algorithm	Phase III	Parallelism	OBLIEXPAND calls	Work W	Span S	Fixed-p time T_p
ObliYan [17]	Kept	Sequential	$\Theta(m)$	$O(m(\Lambda+N)L)$	$\Theta(m(\Lambda+N)L)$	$\Theta(W)$
ParYan (Section V-A2)	Kept	Operator-level	$\Theta(m)$	$O(m(\Lambda+N)L)$	$O(D(\mathcal{T})L)$	$O(W/p + D(\mathcal{T})L)$
JFYan (Section V-B)	**Eliminated**	**Branch-level**	**1**	$\mathbf{O((kN+m\Lambda)L)}$	$\mathbf{O(dL)}$	$\mathbf{O(W/p + dL)}$

oblivious semi-join with aggregation POSJAGG, presented in Section III, covers the reduction phases. Substituting both, we construct an operator-parallelized Yannakakis, ParYan, as our stepping stone and baseline.

This stepping stone, however, leaves the dominant bottleneck untouched. The second obstacle lies *between* operators rather than inside them. Phase III is a chain of oblivious binary joins in which each step consumes the output of the previous one, so the steps run serially across sibling branches; operator-level parallelism cannot break this dependency. Worse, every step re-materializes its intermediate at the output size and re-invokes the most expensive primitive, OBLIEXPAND, giving $\Theta(m)$ output-size expansions along the chain. And expansion is exactly the primitive one should not repeat: to emit 10^6 rows on $p = 16$ cores, OBLIEXPAND alone takes $\approx$210 ms in our implementation, more than OBLISORT ($\approx$144 ms) and over an order of magnitude above the other primitives (Section VII). Thus the bottleneck survives ParYan's operator-level parallelism, motivating the central question of this paper:

Can we evaluate an acyclic join obliviously and *in parallel across the join tree by eliminating the serial materialization chain altogether, rather than parallelizing it operator by operator?*

A. Our Results

We answer this question affirmatively. Our starting point is a simple observation: *counting* the output and *materializing* it are different tasks, and only the latter forces Phase III to be serial. Suppose, instead, that every tuple could learn in advance how many output rows it contributes and exactly which output positions those rows occupy; then the result could be written directly, with no intermediate join ever formed. This reframes acyclic join evaluation from *join materialization* to *position propagation*: a bottom-up pass computes each tuple's output contribution and a top-down pass hands each subtree exactly the slots it owns. Because sibling subtrees receive their slots independently, they can be processed in parallel, and the dependency span shrinks from the $\Theta(m)$-step chain to the join-tree depth d. We call this the *join-free* Yannakakis framework.

Realizing this idea obliviously is not immediate: it must incur no matching-structure leakage, and the difficulty again lies in position assignment. In a multi-ary join tree, a child tuple's output interval depends on its sibling multiplicities; in a multi-level tree, it further depends on the positions inherited from all of its ancestors. Worse, propagating these positions naively reintroduces leakage, exposing per-key group sizes, subtree contributions, and sibling-combination counts.

We resolve these challenges with a *merge–sort–duplicate–compact* mechanism. A bottom-up pass computes each tuple's subtree contribution, and a single oblivious expansion at the root reserves the τ output slots as *position records*. The top-down pass then carries these records down one level at a time: each record converts its global output position into a local coordinate (from the base and stride it inherited), and at a branching node that coordinate picks which child tuple it should inherit. Each step realizes this obliviously by first merging the records with one child relation and then sorting the merged table so that every child tuple sits just before the records that picked it, duplicating the child's data onto them, and compacting away the consumed child-relation tuples, leaving the records (now extended with that data) as the input for the next level. Thus output *positions*, not intermediate joins, propagate down the tree.

Table I compares the resulting complexity of ObliYan, ParYan, and JFYan; our contributions are summarized as follows.

- We propose JFYan, a parallel oblivious *join-free* Yannakakis framework for acyclic joins: it computes subtree contributions bottom-up by POSJAGG, fixes all output positions with a single root expansion, and propagates them top-down through the join tree, reducing the number of output-size OBLIEXPAND calls from $\Theta(m)$ to one and exposing branch-level parallelism across sibling subtrees.
- We show that JFYan is target-adaptive: given any public materialization target $\Lambda \geq \tau$, the same framework runs without revealing the exact output size, instantiating *FO*, *a-FO*, and *DO*. Under all settings, JFYan runs in $O\big((kN + m\Lambda)\log^2(\Lambda + N)/p + d\log^2(\Lambda + N)\big)$ time on p cores, replacing ObliYan's sequential $\Theta\big(m(\Lambda+N)\log^2(\Lambda+N)\big)$.
- We conduct a comprehensive evaluation on TPC-H and TPC-DS. On TPC-DS Query 85 at scale factor 30, with output size 1,977,620 and $p = 16$ cores, our operator-parallelized baseline ParYan runs in 129.75 seconds and achieves a $6.59\times$ speedup over ObliYan, while JFYan runs in 38.13 seconds and achieves a $22.44\times$ speedup over ObliYan. The component breakdown further shows that JFYan's position-propagation part is $8.35\times$ faster than the corresponding Phase II and Phase III materialization stages in ParYan.

II. PROBLEM FORMULATION

A. Problem Definitions

1) Semi-join and Semi-join with aggregation: We define the following semi-join operator and aggregation variant.

Definition II.1 (Semi-join (with aggregation))**.** The semi-join $R \ltimes S = \{r \in R \mid \exists\, s \in S : r.key = s.key\}$ filters R to tuples with at least one match in S [30]. The semi-join with aggregation generalizes this by attaching to each surviving tuple r the aggregate $\bigoplus_{s \in S,\, s.key = r.key} s.val$, where $\bigoplus$ is any operator (e.g., Sum, Count): Let $S_r = \{s \in S \mid s.key = r.key\}$. Then

$$R \ltimes_{\oplus} S = \{(r, \bigoplus_{s \in S_r} s.val) \mid r \in R,\ S_r \neq \emptyset\}. \tag{1}$$

2) Join tree and Acyclic join: We consider a natural join query of the form $Q = R_1 \bowtie R_2 \bowtie \cdots \bowtie R_m$, where each R_i is a relation with attribute set $\mathcal{A}_i$. Let $N = \sum_{i=1}^{m} |R_i|$ denote the total input size, and let $\tau = |Q|$ denote the output size.

Definition II.2 (Join tree)**.** A join tree for Q is a tree $\mathcal{T}$ whose nodes are the relations $R_1, \ldots, R_m$, such that for every attribute $\mathcal{A}$, the set of relations containing $\mathcal{A}$ forms a connected subtree of $\mathcal{T}$ [26].

Definition II.3 (Acyclic join)**.** A natural join query Q is acyclic if it admits a join tree; otherwise, it is cyclic [29].

Throughout this paper, we assume Q is acyclic and fix a rooted join tree $\mathcal{T}$ for Q. For each edge of $\mathcal{T}$, we refer to its two endpoints as a parent v and a child c, with corresponding relations R_v and R_c joined on the shared attributes $\mathcal{A}_v \cap \mathcal{A}_c$. We write $child(v)$ for the set of children of v, $k = \max_v |child(v)|$ for the maximum number of children of any node, and d for the depth of $\mathcal{T}$. We further define the Yannakakis materialization depth of $\mathcal{T}$ as $D(\mathcal{T}) = \max_P \sum_{v \in P} |child(v)| \leq k \cdot d$, where P ranges over root-to-leaf paths of $\mathcal{T}$.

3) Oblivious join: We next formalize the notion of obliviousness that the join algorithm is designed to satisfy.

Definition II.4 (Oblivious join)**.** We denote by $\mathcal{L}(Q, \mathcal{T})$ the leakage function that captures the information observable to the adversary $\mathcal{A}$ during query execution under the threat model [21], [22]. We define two games:

- **Real Game**: The query Q is executed over $\mathcal{T}$ using the real join scheme Π. In this game, the adversary $\mathcal{A}$ can observe the real view of the memory access patterns during query execution.
- **Ideal Game**: The simulator $\mathcal{S}$ takes the leakage $\mathcal{L}(Q, \mathcal{T})$ as input and generates a simulated view of the memory access patterns.

We say that the join scheme Π achieves obliviousness if there exists a probabilistic polynomial-time (PPT) simulator $\mathcal{S}$ such that, for any PPT adversary $\mathcal{A}$, the view of $\mathcal{A}$ in the real execution is computationally indistinguishable from the view generated by $\mathcal{S}$ given only the leakage function $\mathcal{L}(Q, \mathcal{T})$. Formally, we require:

$$\Big| \Pr\left[\mathcal{A}(\textsc{Real}_{\Pi}(Q, \mathcal{T})) = 1\right] - \Pr\left[\mathcal{A}(\textsc{Ideal}_{\mathcal{S}}(\mathcal{L}(Q, \mathcal{T}))) = 1\right] \Big| \leq \mathsf{negl}(\lambda) \tag{2}$$

where λ denotes the security parameter, and $\mathsf{negl}(\lambda)$ is a negligible function of λ.

The model is fixed by the public size in $\mathcal{L}$: *FO* includes the exact output size τ, whereas the output-size-hiding models *a-FO* and *DO* replace it with a τ-independent public bound $\Lambda \geq \tau$, so τ stays hidden.

With these notions in place, we can now state the problem this paper solves.

Problem II.5 (Parallel Oblivious Acyclic Join)**.** Let $Q = R_1 \bowtie \cdots \bowtie R_m$ be an acyclic join with rooted join tree $\mathcal{T}$, with all relations held inside the enclave, and let $\Lambda \geq \tau$ be a public materialization target. We seek a parallel algorithm that writes the result of Q into an output array of public length Λ (its τ result tuples, the remaining $\Lambda - \tau$ slots filled with dummies) such that:

1) **Correctness.** the non-dummy entries are exactly the tuples of Q;
2) **Obliviousness.** it is oblivious in the sense of Definition II.4 with leakage $\mathcal{L}(Q, \mathcal{T}) = (\mathcal{T}, |R_1|, \ldots, |R_m|, \Lambda)$; setting $\Lambda = \tau$ realizes *FO*, while a τ-independent public Λ realizes *a-FO* ($\Lambda = N^{\rho}$) or *DO* ($\Lambda = \Lambda_{\mathrm{DO}}$);
3) **Parallel efficiency.** on p cores it attains small span, ideally governed by the join-tree depth d rather than the number of relations m or the branching factor k.

B. System Model and Threat Model

1) System Model: We consider a TEE-based encrypted database deployed on an untrusted server. The client encrypts the database before outsourcing and provisions the decryption key only to an attested enclave via a secure channel. Query processing is performed inside the enclave.

We focus on a single-enclave shared-memory parallel setting, where the query is executed within one attested enclave over a common address space, as opposed to distributed multi-enclave settings [31]–[33]. Accordingly, p denotes the number of cores used by the parallel execution in our complexity analysis.

2) Threat Model: We adopt the standard threat model used in prior TEE-based oblivious query processing work [15], [34].

- The adversary cannot break TEE hardware protections or access the secret key within the enclave. Meanwhile, physical attacks, denial-of-service attacks, and hardware side-channel attacks (e.g., power analysis [35], timing attacks [13]) are out of scope, as they are orthogonal to our algorithmic focus.
- The adversary observes the join algorithm's memory access patterns, including accessed addresses, access order, access frequency, and data-dependent variation in intermediate result sizes. If any of these depend on the underlying data, the adversary may infer sensitive information such as the number of surviving tuples, per-key group sizes, or join multiplicities. Our goal is therefore to ensure that execution reveals no information beyond $\mathcal{L}(Q, \mathcal{T})$ [21], [22].

C. Oblivious Primitives

We review the oblivious primitives used throughout the paper. For each parallel primitive, we distinguish its work W from its span S and use Brent's bound $T_p = O(W/p + S)$ on p cores [36].

1) Oblivious Sort: The oblivious sort takes an input array X of n elements and produces a sorted permutation by executing a fixed, data-independent sequence of compare-and-swap operations. We adopt bitonic sort [37] as the underlying algorithm, which has work $O(n\log^2 n)$ and span $O(\log^2 n)$, and therefore runs in $O(n\log^2 n/p+\log^2 n)$ time. We denote it as $\textsc{ObliSort}(X)$.

2) Oblivious Expand: The oblivious expand takes an input array X of l tuples each annotated with a repetition count c_i, and outputs an array of length $n=\sum_i c_i$ in which each tuple t_i appears exactly c_i times [13]. It first computes an oblivious prefix sum over the counts to assign each copy a fixed target index, then distributes all copies to their destinations via compare-and-swap in $\log n$ rounds. This gives work $O(n\log n)$, span $O(\log n)$, and $O(n\log n/p+\log n)$ time on p cores [15]. We denote it as $\textsc{ObliExpand}(X,C)$.

3) Oblivious Aggregate: The oblivious aggregate takes an array X of n keyed tuples and a tag-bit array B, where $B[i]=0$ indicates that position i is the first element of a new segment and $B[i]=1$ indicates that position i continues the current segment. It outputs an aggregate value for each element over the segment it belongs to. To compute these aggregates without revealing group boundaries or per-group sizes, the algorithm organizes X into a complete binary tree, and proceeds in two data-independent passes: an upstream pass that propagates partial aggregates toward the root, and a downstream pass that distributes the final values back to the leaves. The algorithm supports three aggregation modes (PREFIX, SUFFIX, and FULL) over two operators, summation and duplication. We use PREFIX in the *exclusive* sense throughout: it returns, for each element, the aggregate of the elements strictly preceding it in its segment (SUFFIX is symmetric). It has work $O(n)$, span $O(\log n)$, and $O(n/p+\log n)$ time on p cores [15]. We denote it as $\textsc{OAggTree}_{\langle \mathsf{type},\mathsf{op}\rangle}(X,B)$.

4) Oblivious Compact: The oblivious compact takes an input array X of n elements together with a mask array $B\in\{0,1\}^n$ of public length [38]. It outputs an array of length n in which all entries with $B[i]=1$ are placed contiguously at the front (the rest filled with dummies), via a fixed sequence of compare-and-swap rounds whose schedule depends only on n. It has work $O(n\log n)$, span $O(\log n)$, and $O(n\log n/p+\log n)$ time on p cores [39]. We denote it as $\textsc{ObliCompact}(X,B)$.

5) Parallel Oblivious Two-Way Join: The oblivious two-way join takes two relations R and S and outputs their equi-join result $R\bowtie S$ while hiding the matching structure beyond the output size. It proceeds in three steps: (i) OAGGTREE computes per-key counts $c_R(key)$ and $c_S(key)$ in both relations; (ii) OBLIEXPAND replicates each tuple $r\in R$ by $c_S(r.key)$ times and symmetrically for S; (iii) both expanded arrays are aligned by an OBLISORT, after which a linear scan materializes the result [12], [13]. Let $n=|R|+|S|+|R\bowtie S|$ denote the combined input and output size, since steps (ii)–(iii) operate on the materialized join. The work is $O(n\log^2 n)$ and the span is $O(\log^2 n)$, so the p-core running time is $O(n\log^2 n/p+\log^2 n)$ [15]. We denote it as $\textsc{POTWJ}(R,S)$.

III. PARALLEL OBLIVIOUS SEMI-JOIN WITH AGGREGATION

This section presents POSJAGG, a parallel oblivious semi-join with aggregation that serves as the reduction operator of both algorithms: ParYan invokes it in place of the serial semi-join in Phases I and II, and JFYan invokes it in the bottom-up contribution computation (Section V). We define the operator as a semi-join *with aggregation* (Definition II.1) rather than a plain semi-join, because obliviousness rules out deletion: an oblivious semi-join cannot physically delete a non-matching tuple, since deletion would leak which tuples matched; it can only overwrite the tuple with a dummy in a fixed-length array, and writing a dummy is exactly assigning the tuple the additive identity 0 of an aggregation. Oblivious realizations of this operator already exist: both Arasu [12] and Hu [17] sort on the join key and then sweep the sorted array with a sequential linear scan. That scan, however, is the obstacle to parallelism: each tuple's aggregate depends on the running state of the preceding tuples sharing its key, which forces the computation to proceed strictly in order. We remove this dependency by replacing the linear scan with a segmented tree aggregation, yielding $\textsc{POSJAgg}(R,S)$, a parallel oblivious semi-join with aggregation built entirely from parallel oblivious primitives (Algorithm 1). Throughout this section, $n=|R|+|S|$ denotes the total input size.

Algorithm 1 Parallel Oblivious Semi-Join with Aggregation

Require: Relations $R(key,val)$, $S(key,val)$
Ensure: $R\ltimes S$ with aggregation over the val
▷ Step 1: Tag and merge
assign row number $idx(t)$ to each tuple $t\in R$ and $t\in S$
$\mathcal{M}\leftarrow\big[(r.(key,val,idx),\ tid{=}R):r\in R\big]\cup\big[(s.(key,val,idx),\ tid{=}S):s\in S\big]$
▷ Step 2: Oblivious sort by $(key,\ tid)$ with $S<R$
$\mathcal{M}\leftarrow\textsc{ObliSort}\big(\mathcal{M},\ (key,\ tid)\big)$
▷ Step 3: Encode
for each $i\in[0,n)$ **in parallel do**
  $B[i]\leftarrow\mathcal{M}[i].key=\mathcal{M}[i{-}1].key$
end for
▷ Step 4: Compute segmented prefix aggregation
$Sc\leftarrow\textsc{OAggTree}_{\langle \mathsf{prefix},\oplus\rangle}\big(\mathcal{M}.val\cdot(tid{=}S),B\big)$
▷ Step 5: Attach aggregates and restore tuple order
$\mathcal{M}\leftarrow\textsc{ObliSort}(\mathcal{M},(tid,idx))$
return R and S annotated with their aggregate values.

In Step 2, the merged table is sorted so that all S-tuples precede the corresponding R-tuples within each key group. Consequently, the segmented *exclusive*-prefix aggregation in Step 4 has accumulated the full S-side aggregate by the time any R-tuple is reached, delivering $Sc[i]=\bigoplus_{s\in S(r)}s.val$ at every R-position without a sequential scan over matching tuples. The same pass simultaneously gives each $s\in S$ the cumulative aggregate of the S-tuples strictly preceding it in its key group, which serves directly as the rank used later in the join-free Yannakakis algorithm (Section V).

Theorem III.1 (POSJAGG properties)**.** *Algorithm 1 produces both R and S annotated: each $r\in R$ receives the matching*

aggregate $\bigoplus_{s\in S(r)} s.val$*, where* $S(r) = \{s \in S : s.key = r.key\}$*; each* $s \in S$ *receives the cumulative aggregate over the* S*-tuples strictly preceding it in its key group. It runs in* $O(n \log^2 n/p + \log^2 n)$ *time. Assuming* OBLISORT *and* OAGGTREE *are oblivious, the algorithm is oblivious with leakage* $\mathcal{L}_{\text{POSJAGG}}(R, S) = (|R|, |S|)$.

Proof sketch. The sorting invariant of Step 2 guarantees that each R-tuple is preceded by exactly its matching S-tuples, so the prefix sum in Step 4 yields the correct per-tuple aggregate (matching aggregate on the R side, cumulative-prefix aggregate on the S side); Step 5 then restores the original input order. These two OBLISORT calls dominate, giving $O(n \log^2 n/p + \log^2 n)$ time. For obliviousness, every remaining operation is either a fixed-pattern scan of length n or an invocation of an oblivious primitive on a public-size input, so the access trace depends only on $(|R|, |S|)$ and can be simulated by composing the OBLISORT and OAGGTREE simulators on inputs of length n. □

IV. JOIN-FREE YANNAKAKIS ALGORITHM

We propose a *join-free* variant of the Yannakakis algorithm that exploits branch-level parallelism by eliminating Phase III and restructuring the computation into two parallel-friendly phases. For exposition, this section describes the non-oblivious idea and identifies its access pattern leakages; the oblivious counterpart is presented in Section V.

A. Classical Yannakakis Algorithm

The classical Yannakakis algorithm evaluates an acyclic join Q in $O(N + \tau)$ through three phases [23], [26].

1) **Phase I (Bottom-up semi-join reduction).** It traverses the join tree bottom-up. When node c is visited in this order, its parent v is updated to $R_v \leftarrow R_v \ltimes R_c$.
2) **Phase II (Top-down semi-join reduction).** It traverses the join tree top-down. When node v is visited, each child c of v is updated to $R_c \leftarrow R_c \ltimes R_v$.
3) **Phase III (Bottom-up join materialization).** It traverses bottom-up to materialize the full join result. When node v with children $c_1, \cdots, c_k$ is visited, R_v is iteratively updated as $R_v \leftarrow R_v \bowtie R_{c_j}$ for each $j \in [1, k]$.

Parallelism analysis. In the join tree, for example, R_1 serves as the root and has two child relations, R_2 and R_3. For Phases I and II, the filtering condition for each parent tuple $t = (a, b) \in R_1$ decomposes as:

$$t \in \{(a, b) \in R_1 \mid a \in \pi_A(R_2) \wedge b \in \pi_B(R_3)\} \tag{3}$$

where the two conditions are independent and can be evaluated concurrently across sibling subtrees, admitting natural branch-level parallelism. Phase III does not admit such a decomposition. For example, before R_1 can be joined with R_3, it must first be joined with R_2, with the intermediate result assigned back to R_1, and then joined with R_3 in the same manner. The sequential dependencies among sibling joins prevent branch-level parallelism and constitute the primary bottleneck.

B. Join-free Yannakakis Algorithm

Phase III is serial and expensive precisely because it repeatedly *materializes* joins. We therefore avoid materialization entirely: a bottom-up pass counts how many output rows each tuple will contribute, and a top-down pass then hands each tuple its output positions directly. These two phases, contribution computation and position propagation, together replace the classical three.

1) Bottom-up semi-join with contribution computation: The bottom-up pass traverses the join tree from the leaves upward. Each leaf initializes $\Delta_c[t] = 1$. At each internal node v with children $c_1, \ldots, c_k$, for every $t \in R_v$:

1) **Subtree contribution.** For each child c, sum the contributions of matching tuples: $u_c^v[t] = \sum_{s_q \in R_c(t)} \Delta_c[s_q]$, where $R_c(t) = \{s \in R_c : s.key = t.key\}$.
2) **Total contribution.** Combine across children by product: $\Delta_v[t] = \prod_{c \in \text{child}(v)} u_c^v[t]$. Tuple t survives iff $\Delta_v[t] > 0$.

At the root, a prefix sum over $\{\Delta_{root}[t]\}$ assigns each surviving tuple a base offset $b[t]$ and disjoint output interval $[b[t], b[t] + \Delta_{root}[t] - 1]$.

2) Top-down semi-join with position assignment: The top-down pass traverses the join tree from the root downward, recursively propagating and partitioning intervals from parent to children. We address three sub-problems:

1) **Simultaneous filtering.** We sort $\mathcal{M} = R_v \cup R_{c_j}$ by join key, placing parent tuples before child tuples within the same key, and maintain a lookup table $T[key]$ that records the parent-side interval statistics for each key. During the scan, when a parent tuple $t \in R_v$ is encountered, its interval state is inserted into $T[t.key]$; when a child tuple $s \in R_{c_j}$ is encountered, its interval is read from $T[s.key]$ for position assignment, and it is discarded if $T[s.key]$ is empty (no matching parent).
2) **Multi-ary tree assignment.** For a parent tuple $t \in R_v$, its output interval has size $\Delta_v[t] = \prod_j u_{c_j}^v[t]$, representing the Cartesian product of all child matches. Before selecting the R_{c_j} component, the choices for its left siblings account for $\prod_{\ell<j} u_{c_\ell}^v[t]$ combinations. Within each such combination, R_{c_j} is divided into $u_{c_j}^v[t]$ sub-intervals of length $\sigma_{c_j}[t] = \prod_{\ell>j} u_{c_\ell}^v[t]$, one for each matching tuple in R_{c_j}. Specifically, the i_{c_j}-th matching tuple $s_{i_{c_j}} \in R_{c_j}(t)$ (for $i_{c_j} \in [0, u_{c_j}^v[t] - 1]$) is assigned the sub-interval $[l(s_{i_{c_j}}), r(s_{i_{c_j}})]$ given by

$$\begin{aligned} l(s_{i_{c_j}}) &= b[t] + \textstyle\sum_{\ell<j} i_{c_\ell} \cdot \sigma_{c_\ell}[t] + i_{c_j} \cdot \sigma_{c_j}[t], \\ r(s_{i_{c_j}}) &= l(s_{i_{c_j}}) + \sigma_{c_j}[t] - 1. \end{aligned} \tag{4}$$

3) **Multi-level tree assignment.** Propagating the multi-ary rule recursively raises two issues. *(i) Vertical mismatch.* When R_{c_j} is not a leaf, $u_{c_j}^v[t]$ no longer counts the tuples in R_{c_j} that directly match t; instead, it equals the total output contribution of the subtree rooted at c_j, namely $u_{c_j}^v[t] = \sum_{s_q \in R_{c_j}(t)} \Delta_{c_j}[s_q]$. Hence each matching tuple s_q no longer occupies a single stride-length slot but a longer block of length $\Delta_{c_j}[s_q] \cdot \sigma_{c_j}[t]$, proportional to its subtree

weight. Concretely, the cumulative offset of s_q within R_{c_j} becomes $rank_{c_j}(s_q) = \sum_{r<q} \Delta_{c_j}[s_r]$, and Eq. (4) applies with i_{c_j} replaced by $rank_{c_j}(s_q)$. *(ii) Horizontal mismatch.* Even after (i) is fixed, the position computed by Eq. (4) lives in the local coordinate system of the current subtree, not in the global output. The missing piece is the cumulative stride contributed by every right-sibling cross product along the path from the root to c_j. We track this by augmenting each inherited state to a pair $(b[t], \lambda)$, where λ accumulates ancestor strides as $\lambda_{c_j} = \lambda_v \cdot \sigma_{c_j}[t]$ (with $\lambda = 1$ at the root). The effective stride at level c_j then becomes $\lambda \cdot \sigma_{c_j}[t]$, replacing $\sigma_{c_j}[t]$ everywhere in Eq. (4).

Dependency on left siblings. The above procedure carries a sequential dependency: although the k children of a parent can each be processed in parallel, the position assigned to any tuple in R_{c_j} still depends on the indices $i_{c_1}, \ldots, i_{c_{j-1}}$ of all its left siblings (the $\sum_{\ell<j} i_{c_\ell} \cdot \sigma_{c_\ell}[t]$ term in Eq. (4)). We resolve this dependency in the oblivious version (Section V).

C. Access-Pattern Leakages

The *join-free* variant introduces new challenges in the oblivious setting. We identify three sources of access-pattern leakage and explain why naive padding fails to address them.

1) $\mathcal{L}_1$ **(Child-side leakage).** The first source of leakage comes from child-side group processing. Computing $u^p_c[t]$ requires aggregating over the matching set $R_c(t)$; computing $rank_{c_j}(s_q)$ requires summing preceding tuples in the same child-side key group; and discarding unmatched child tuples reveals whether a match exists. Therefore, the trace reveals not only whether a child tuple has a matching parent tuple, but also how many tuples in R_{c_j} share the same join key, i.e., the size of $R_{c_j}(t)$ and its finer within-group structure.
2) $\mathcal{L}_2$ **(Parent-side leakage).** The second source of leakage comes from parent-side state lookup. For each child tuple, a direct implementation looks up the parent group sharing its join key and scans all entries in that group to read their interval state; the number of entries scanned equals the per-key parent count. As a result, the occupancy of each key group becomes observable, revealing how many parent tuples share the same join key.
3) $\mathcal{L}_3$ **(Propagation leakage).** The third source of leakage arises from top-down propagation itself. In a multi-level tree, a single $t \in R_p$ may be reached by several distinct ancestor combinations, each contributing its own inherited (b, λ) pair; we denote the resulting multiset by $state_p[t]$. A direct implementation iterates over every inherited state in $state_p[t]$ and every coordinate in $W_{c_j}(t)$, where $|W_{c_j}(t)| = \prod_{\ell<j} u^p_{c_\ell}[t]$, and then appends one interval and one child state for each such pair. Consequently, the number of append operations depends on both $|state_p[t]|$ and $\prod_{\ell<j} u^p_{c_\ell}[t]$. The access pattern therefore exposes both $|state_p[t]|$ and $\prod_{\ell<j} u^p_{c_\ell}[t]$, namely the number of inherited states and the number of left-sibling combinations.

Why naive padding fails. A natural fix is to pad every data-dependent loop to a fixed public worst-case length. For each R_{c_j}, determining its starting position requires iterating over the left-sibling multiplicities $u^p_{c_1}[t], \ldots, u^p_{c_{j-1}}[t]$. To hide this leakage, each $u^p_{c_\ell}[t]$ must be padded to $U_{c_\ell} = \prod_{R_w \in \mathcal{T}_{c_\ell}} |R_w|$, which depends only on public relation sizes and the public join tree. Consequently, assigning positions for the j-th child costs $O(\prod_{\ell<j} U_{c_\ell})$ per parent tuple, and the overall procedure degenerates into Cartesian-product enumeration over the child subtrees, which renders full obliviousness impractical.

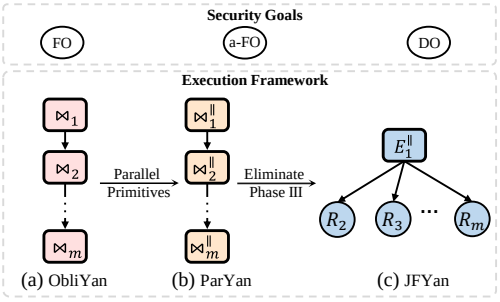


Fig. 1: Different ways of handling Phase III under different obliviousness goals.

V. Parallel Oblivious Algorithm for Acyclic Join

In this section, we present a parallel oblivious algorithm for acyclic join based on the *join-free* Yannakakis framework. We develop the algorithm under the *FO* model, where the single root expansion runs at the exact output size τ; its *a-FO* and *DO* instantiations, obtained by changing only the public materialization target, are introduced in Section V-C, the security and efficiency are analyzed uniformly in Section VI.

A. From ObliYan to Our Algorithm

We first recall the state-of-the-art and optimal baseline ObliYan, then introduce ParYan as an intermediate parallelized baseline that motivates and isolates the contribution of our final algorithm JFYan (presented in Section V-B). A summary is shown in Figure 1.

1) Baseline (ObliYan): ObliYan instantiates Yannakakis with oblivious operators. For Phases I and II, each edge (v, c) runs ObliSemiJoin at cost $O((|R_v|+|R_c|)\log^2(|R_v|+|R_c|))$. Summing over all edges, each semi-join phase costs $O\big(\sum_{(v,c)\in E(\mathcal{T})}(|R_v| + |R_c|)\log^2(|R_v| + |R_c|)\big)$; since each relation participates in at most $k+1$ edges of $\mathcal{T}$, this is bounded by $O(kN \log^2 N)$. In Phase III, ObliYan materializes the output by bottom-up oblivious binary joins. At each edge (v, c), the current intermediate at v is padded to the final output size τ and then joined with R_c, costing $O((\tau + |R_c|)\log^2(\tau + |R_c|))$ per step, so summing over the $m-1$ edges gives $O(m\cdot(\tau+N)\log^2(\tau+N))$, which dominates the total cost of ObliYan. Since every step depends on the previous one, the computation is fully sequential, running in $O(m(\tau + N)\log^2(\tau + N))$ time regardless of p.

2) Parallelized baseline (ParYan): ParYan is an operator-parallelized variant of ObliYan: it replaces the sequential oblivious primitives in ObliYan with the same parallel counterparts used by JFYan. For Phases I and II, substituting ObliSemiJoin with our POSJAGG (Section III) lets sibling

subtrees be scheduled concurrently over the shared thread pool, running in $O(kN\log^2(\tau+N)/p + d\log^2(\tau+N))$ (these phases touch only the input relations, so $\log^2 N$ suffices; we keep $\log^2(\tau+N)$ for a bound uniform with Phase III). For Phase III, substituting OBLITWOWAYJOIN with POTWJ parallelizes each binary join internally, but does not remove the materialization dependency, whose critical path still contains $D(\mathcal{T})$ sequential binary-join steps. Overall, on p cores ParYan runs in $O\big(m(\tau+N)\log^2(\tau+N)/p + D(\mathcal{T})\log^2(\tau+N)\big)$: each operator is internally parallelized, but the $\Theta(m)$ output-size expansions and the Phase III chain both remain. Removing this expansion chain is exactly what JFYan in Section V-B achieves.

Algorithm 2 Oblivious Bottom-up Contribution Computation

Require: Join tree $\mathcal{T}$, relation set $\{R_i : R_i \in \mathcal{T}\}$
Ensure: u_c^v and $rank_c$ computed for each tuple; dangling tuples removed; Δ assigned to each tuple; root base offset b and $state$ assigned; copy table C of R_{root} with pos on each copy

▷ Step 1: Bottom-up semi-join and compute Δ_c and u_c^v
for each node $R_c \in \mathcal{T}$ in **bottom-up** order **do**
  for each $i \in [0, |R_c|)$ **in parallel do**
    $R_c[i].\Delta \leftarrow \prod_{w:\text{ child of } R_c} R_c[i].u_w^c$
    if $R_c[i].\Delta = 0$ **then**
      $R_c[i] \leftarrow \perp$
    else
      $R_c[i] \leftarrow R_c[i]$
    end if
  end for
  if R_c has parent R_v **then**
    $(R_v,\ u_c^v,\ R_c.rank_c) \leftarrow \text{POSJAGG}(R_v,\ R_c)$
  end if
end for
▷ Step 2: Assign base offset and initialize root state
$b \leftarrow \text{OAGGTREE}_{\langle \text{prefix},+\rangle}\big(R_{root}.\Delta,\ \mathbf{1}^{|R_{root}|}\big)$
for each $i \in [0, |R_{root}|)$ **in parallel do**
  $R_{root}[i].b \leftarrow b[i],\ R_{root}[i].\lambda \leftarrow 1$
  $R_{root}[i].state \leftarrow (R_{root}[i].b,\ R_{root}[i].\lambda)$
end for
▷ Step 3: Expand R_{root} by Δ to enumerate all output positions
$C \leftarrow \text{OBLIEXPAND}(R_{root},\ \Delta)$
for each $i \in [0, |C|)$ **in parallel do**
  $C[i].pos \leftarrow i$
end for
return join tree $\mathcal{T}$ and root copy table C

B. Our Main Algorithm: JFYan

We now present our final algorithm JFYan, which eliminates Phase III altogether and thereby removes the $\Theta(m)$ OBLIEXPAND chain in ParYan. Figure 2 illustrates the resulting process on the running example. For clarity, we only show the propagation along the left branch.

1) Oblivious bottom-up semi-join with contribution computation: The procedure traverses the join tree bottom-up. At each internal node v with children $c_1, \ldots, c_k$:

1) **Subtree contribution.** It invokes POSJAGG to compute $u_c^v[t]$ for each tuple $t \in R_v$ and each child $c \in \text{child}(v)$.

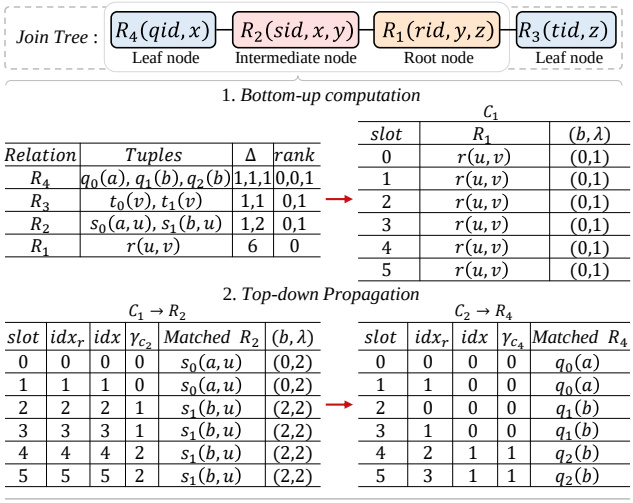


Fig. 2: JFYan on the running example.

2) **Total contribution.** It computes the multiplicity of each tuple $t \in R_v$ as $\Delta_v[t] = \prod_{c\in\text{child}(v)} u_c^v[t]$. If $\Delta_v[t] = 0$, the tuple is replaced with $\perp$; otherwise, it is retained.

At the end of the above phase, the total output size $\Delta_{\text{root}}[t]$ of every surviving root tuple $t \in R_{\text{root}}$ is fully determined. We invoke OAGGTREE in prefix-sum mode on $\{\Delta_{\text{root}}[t]\}$ to compute a base offset $b[t]$ for each surviving tuple, so that t occupies the contiguous output interval $[b[t],\ b[t]+\Delta_{\text{root}}[t]-1]$. We then call OBLIEXPAND *once* to replicate every surviving root tuple t into its assigned interval, producing the copy table C of length $\tau = \sum_t \Delta_{\text{root}}[t]$. This is the *only* expansion in the entire algorithm; every subsequent top-down step merely redistributes these τ positions at the fixed public length τ, creating no new ones. Each entry $x \in C$ represents one output row and is initialized with three fields: $x.b = b[t]$ (the base offset inherited from its root tuple), $x.pos$ (the global output position of this specific copy, equal to its row number in C), and $x.\lambda = 1$ (the accumulated stride product from ancestor levels; initialized to 1 at the root and multiplied by the current child stride σ_{c_j} at each level, allowing local-coordinate recovery from the global copy index without re-expansion). The procedure is shown in Algorithm 2.

2) Oblivious top-down semi-join with position assignment: To assign output positions we address three subproblems: (1) discarding dangling tuples, (2) matching each copy to its child tuple, and (3) extending both to a multi-level tree. As Algorithm 3 shows, (1) and (2) are not separate traversals: a single merge–sort(OBLISORT)–duplicate(OAGGTREE)–compact(OBLICOMPACT) pass per edge matches copies to child tuples and removes dangling tuples at once.

(1) Simultaneous filtering without leakage. Filtering is not a separate step: dangling tuples are removed by the same matching pass described in (2) below (its closing OBLICOMPACT), so we never branch on whether a child tuple has a matching parent.

(2) Multi-ary tree assignment without leakage. After sorting C_v by $(state, pos)$, we assign each copy x a group-local rank

idx_r via an oblivious prefix sum. Dividing by λ gives the current-node coordinate idx, which is then decomposed into the per-child coordinate:

$$x.\gamma_{c_j} = \left\lfloor x.idx / x.\sigma_{c_j} \right\rfloor \bmod u^v_{c_j}[t]. \tag{5}$$

This coordinate selects a child-side contribution coordinate: it is the rank of the selected tuple for a leaf child, as made explicit next. To realize this matching obliviously, we merge C_v and R_{c_j} into table $\mathcal{M}$ and sort it by (key, ord). For each $s_q \in R_{c_j}$, we set $ord = rank_{c_j}(s_q)$; for each $x \in C_v$, we set $ord = x.\gamma_{c_j}$. The child tuple is ordered before all copy entries whose ord falls before the next child rank in the same key group, so it becomes the segment header for exactly its contribution interval. We then invoke OAGGTREE in duplication mode to copy the attributes of s_q to the matched copies x. Consequently, each matched copy x retains its original $x.pos$ and obtains the attributes of s_q, enabling the subsequent top-down propagation to the next level. A final OBLICOMPACT then keeps these matched copies and removes all R_{c_j} entries, which simultaneously realizes the filtering of (1).

(3) Multi-level tree assignment without leakage. We employ the following two strategies within a multi-level tree.

a) **Vertical strategy.** In a multi-level tree, $u^v_{c_j}[t]$ no longer denotes the count of matching tuples but the sum of their subtree contributions: $u^v_{c_j}[t] = \sum_{s_q \in R_{c_j}(t)} \Delta_{c_j}[s_q]$. We accordingly redefine the cumulative offset $rank_{c_j}(s_q) = \sum_{r<q} \Delta_{c_j}[s_r]$, which is precomputed obliviously in Algorithm 2 as a key-segmented prefix sum of Δ_{c_j} via OAGGTREE. Thus each s_q spans a range of coordinates $J = \left[rank_{c_j}(s_q),\ rank_{c_j}(s_q) + \Delta_{c_j}[s_q]\right)$ rather than a single point, and a copy x is matched to the unique s_q with $x.\gamma_{c_j} \in J$. The merge–sort–duplicate–compact mechanism of (2) realizes this matching obliviously without modification, using the redefined $rank_{c_j}$ as the child-side ord key.

b) **Horizontal strategy.** While the vertical step determines which child tuple a copy should use, the horizontal step determines where the resulting copy should be placed in the global output table. This requires two changes in the top-down propagation. First, before decomposing a copy into child-side coordinates, we must remove the repetition inherited from ancestors. Copies are sorted by $(state, pos)$, so copies with the same inherited state are grouped in increasing global-output order; this order is needed because the lower-order residual positions represent repetitions already introduced by ancestors and right siblings. An oblivious prefix sum assigns each copy a group-local index idx_r. This index is counted over the expanded copy table, so it includes the λ repetitions already introduced by ancestor levels. Therefore, we compute $idx = \lfloor idx_r / \lambda \rfloor$. The value idx is the current-node coordinate, which the sibling strides can then decompose to obtain γ_{c_j} for each child. Second, after the child tuple's attributes have been propagated, we update the state (b, λ) for the next level. Rather than materializing the left-sibling offset $\sum_{\ell<j} \gamma_{c_\ell} \sigma_{c_\ell}$ explicitly, we use $x.pos$, which already encodes the full global placement including all sibling contributions. We subtract only the two parts that should not propagate to the child: $\alpha = idx \bmod \sigma_{c_j}$ (right-sibling residual) and $\beta = (\gamma_{c_j} - rank_{c_j}(s_q)) \cdot \sigma_{c_j}$ (offset within the matched child's contribution range). Both are in current-node coordinates; multiplying by λ rescales them to the expanded copy table:

$$b = x.pos - \lambda(\alpha + \beta), \qquad \lambda = \lambda \cdot \sigma_{c_j}. \tag{6}$$

The state (b, λ) passed to the child thus encodes the correct base position without ever computing the left-sibling sum directly.

Algorithm 3 Oblivious Top-down Position Assignment

Require: Join tree $\mathcal{T}$ and the root copy table C from Algorithm 2
Ensure: Copy tables whose tuples are assigned final positions

1: **for** each node $R_v \in \mathcal{T}$ in **top-down** order **do**
2: $C_v \leftarrow \text{OBLISORT}(C_v, (state, pos))$
3: **for** each $i \in [0, |C_v|)$ **in parallel do**
4: $B_C[i] \leftarrow \big(C_v[i].state = C_v[i-1].state\big)$
5: **end for**
6: $C_v.idx_r \leftarrow \text{OAGGTREE}_{\langle \text{prefix},+\rangle}\big(\mathbf{1}^{|C_v|}, B_C\big)$
7: **for** each child R_{c_j} of R_v **in parallel do**
8: **for** each $i \in [0, |C_v|)$ **in parallel do**
9: $C_v[i].idx \leftarrow \lfloor C_v[i].idx_r / C_v[i].\lambda \rfloor$
10: $C_v[i].\sigma_{c_j} \leftarrow \prod_{\ell>j} C_v[i].u^v_{c_\ell}$
11: $C_v[i].\gamma_{c_j} \leftarrow \left\lfloor C_v[i].idx / C_v[i].\sigma_{c_j} \right\rfloor \bmod C_v[i].u^v_{c_j}$
12: **end for**
13: $\mathcal{M}_v \leftarrow \{(x.key, x.\gamma_{c_j}, x, R_v) : x \in C_v\}$
14: $\mathcal{M}_c \leftarrow \{(y.key, y.rank_{c_j}, y, R_{c_j}) : y \in R_{c_j}\}$
15: $\mathcal{M} \leftarrow \mathcal{M}_v \cup \mathcal{M}_c$
16: $\mathcal{M} \leftarrow \text{OBLISORT}(\mathcal{M}, (key, ord))$, with $R_{c_j} < R_v$
17: **for** each $i \in [0, |\mathcal{M}|)$ **in parallel do**
18: $B_{R_v}[i] \leftarrow (\mathcal{M}[i].tid = R_v)$
19: $B_{\mathcal{M}}[i] \leftarrow (\mathcal{M}[i].key = \mathcal{M}[i-1].key) \wedge B_{R_v}[i]$
20: **end for**
21: $X := \mathcal{M}.tuple \parallel tid \parallel rank_{c_j}$
22: $\mathcal{M}' \leftarrow \text{OAGGTREE}_{\langle \text{prefix,dup}\rangle}(X,\ B_{\mathcal{M}})$
23: $C_{c_j} \leftarrow \text{OBLICOMPACT}(\mathcal{M}', B_{\mathcal{M}})[\,0 : |C_v|\,]$
24: **for** each $i \in [0, |C_{c_j}|)$ **in parallel do**
25: $\alpha \leftarrow C_{c_j}[i].idx \bmod C_{c_j}[i].\sigma_{c_j}$
26: $\beta \leftarrow \big(C_{c_j}[i].\gamma_{c_j} - C_{c_j}[i].rank_{c_j}\big) \cdot C_{c_j}[i].\sigma_{c_j}$
27: $C_{c_j}[i].b \leftarrow C_{c_j}[i].pos - C_{c_j}[i].\lambda \cdot (\alpha + \beta)$
28: $C_{c_j}[i].\lambda \leftarrow C_{c_j}[i].\lambda \cdot C_{c_j}[i].\sigma_{c_j}$
29: $C_{c_j}[i].state \leftarrow \big(C_{c_j}[i].b, C_{c_j}[i].\lambda\big)$
30: **end for**
31: **end for**
32: **end for**
33: **return** copy tables $\{C_v : v \in \mathcal{T}\}$

C. Target-Adaptive Materialization

The algorithm above is described under *FO*, where the root is expanded to the exact output size τ. Since this root expansion is the only cardinality-increasing materialization step, other output-size targets are supported by changing only its expansion length. Let $\Lambda \geq \tau$ be a public materialization target. Before line 22 of Algorithm 2, we append a marked dummy root tuple $t_\perp$ with contribution count $\Delta[t_\perp] = \Lambda - \tau$.

The root counts then sum to Λ, so ObliExpand directly produces a public length-Λ copy table, without materializing any length-τ intermediate in the trace.

Dummy copies follow the same oblivious propagation trace as real copies and are routed to fixed dummy payloads on child edges. Therefore, the real output positions and tuple assignments remain unchanged, while output-size-hiding modes return a public length-Λ array and let the client discard marked dummy rows after decryption. This single hook instantiates *FO* with $\Lambda = \tau$, *a-FO* with $\Lambda = N^{\rho}$, where ρ is the fractional edge-cover number, and *DO* with a differentially private upper bound Λ_{DO} [18]. Section VI analyzes security and efficiency uniformly for any public target $\Lambda \geq \tau$.

D. Correctness

The correctness follows from the following two lemmas.

Lemma V.1. *Fix an edge (p, c_j) and a copy $x \in C_p$ whose parent tuple is $t \in R_p$, with $\sigma_{c_j} = \prod_{\ell>j} u^p_{c_\ell}[t]$ and $x.p_{c_j} = \lfloor x.idx/\sigma_{c_j} \rfloor \bmod u^p_{c_j}[t]$. Let $R_{c_j}(t) = \{s_1, \ldots, s_r\}$ in the order used by Algorithm 3, and let $I_q = [rank_{c_j}(s_q),\ rank_{c_j}(s_q) + \Delta_{c_j}[s_q])$. Then $I_1, \ldots, I_r$ partition $[0, u^p_{c_j}[t])$, so a unique s_q satisfies $x.p_{c_j} \in I_q$, and the merge–sort–OAggTree step attaches exactly this s_q to x.*

Proof. By definition $u^p_{c_j}[t] = \sum_q \Delta_{c_j}[s_q]$ and $rank_{c_j}(s_q)$ is the prefix sum of preceding Δ_{c_j} within the join-key group, so the I_q are consecutive, disjoint, and cover $[0, u^p_{c_j}[t])$; the extracted $x.p_{c_j}$ lies in a unique I_q. In the merged table sorted by (key, ord), child tuple s_q has $ord = rank_{c_j}(s_q)$ and copy x has $ord = x.p_{c_j}$, so every copy with $x.p_{c_j} \in I_q$ falls between s_q and the next child header. The duplication-mode OAggTree propagates s_q to exactly those copies, which ObliCompact retains; disjointness rules out any other $s_{q'}$. □

Lemma V.2. *After Algorithm 3 processes node $v \in \mathcal{T}$, every copy $x \in C_v$ attached to $t_x \in R_v$ admits a unique local coordinate $idx^{(v)}(x) \in [0, \Delta_v[t_x])$ and residual $\rho^{(v)}(x) \in [0, x.\lambda)$ such that*

$$x.pos = x.b + x.\lambda \cdot idx^{(v)}(x) + \rho^{(v)}(x). \tag{7}$$

Within each fixed state (b, λ), sorting copies by pos makes $\lfloor idx_r/x.\lambda \rfloor = idx^{(v)}(x)$. Thus the algorithm recovers the correct current-node coordinate at every level and preserves the global output position.

Proof. At the root, b is the prefix-sum base offset, $\lambda = 1$, $\rho^{(root)} = 0$, and ObliExpand gives $idx^{(root)}(x) = x.pos - x.b$, so (7) holds. Assume it holds at parent p. Sorting C_p by $(state, pos)$ places same-state copies in increasing global position, so for each local coordinate there are exactly $x.\lambda$ residuals before the next, giving $\lfloor idx_r/x.\lambda \rfloor = idx^{(p)}$ as used in line 9. By Lemma V.1, p_{c_j} selects a unique s_q; let $\delta = p_{c_j} - rank_{c_j}(s_q)$, $\alpha = idx^{(p)} \bmod \sigma_{c_j}$, $\beta = \delta \cdot \sigma_{c_j}$, with mixed-radix decomposition $idx^{(p)} = \sum_{\ell<j} p_{c_\ell}\sigma_{c_\ell} + rank_{c_j}(s_q)\sigma_{c_j} + \beta + \alpha$. The algorithm's updates $b' = x.pos - x.\lambda(\alpha+\beta)$ and $\lambda' = x.\lambda \cdot \sigma_{c_j}$ yield $x.pos = b' + \lambda'\delta + \rho'$ with $\rho' = x.\lambda\alpha + \rho^{(p)}(x) < \lambda'$ (since $\alpha < \sigma_{c_j}$ and $\rho^{(p)}(x) < x.\lambda$). Hence (7) holds at c_j with $idx^{(c_j)}(x) = \delta$, and the global position is preserved. □

VI. Security and Efficiency Analysis

We analyze JFYan for a general public target $\Lambda \geq \tau$, so every result below holds uniformly for all three models (Section V-C).

A. Efficiency Analysis

Table I summarizes the asymptotic costs under the target-adaptive padding rule: every output-bearing copy table has public length Λ, and we set $L = \log^2(\Lambda + N)$. ObliYan and ParYan both perform $O(m(\Lambda+N)L)$ work and invoke ObliExpand $\Theta(m)$ times on output-sized intermediates; consequently, their fixed-p running times are $\Theta(m(\Lambda+N)L)$ and $O(m(\Lambda+N)L/p + D(\mathcal{T})L)$, respectively. By contrast, JFYan performs a single root expansion and exploits branch-level parallelism across sibling subtrees, for overall work $O((kN + m\Lambda)L)$, span $O(dL)$, and fixed-p time $O((kN + m\Lambda)L/p + dL)$.

B. Security Analysis

We prove the obliviousness of the parallel oblivious algorithm for acyclic join under the Real/Ideal paradigm. The leakage function is $\mathcal{L}(Q, \mathcal{T}) = (\mathcal{T}, |R_1|, \ldots, |R_m|, \Lambda)$, where Λ is the public materialization target. The proof is for an arbitrary public target $\Lambda \geq \tau$, hence covers all three models at once. Recall from access pattern leakage IV-C that the join-free template introduces three concrete leakage sources $\mathcal{L}_1$ (child side), $\mathcal{L}_2$ (parent side), and $\mathcal{L}_3$ (propagation). We first show that Algorithms 2 and 3 eliminate each of these sources by construction (Lemma VI.1); the main obliviousness theorem then follows by simulator composition.

Lemma VI.1 (Leakage closure)**.** *Algorithms 2 and 3 eliminate the three leakage sources $\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$ identified in leakage analysis IV-C, in the sense that the access pattern produced by every step that previously induced $\mathcal{L}_i$ now depends only on $\mathcal{L}(Q, \mathcal{T})$.*

Proof. We address the three sources in turn.

1) **Closing $\mathcal{L}_1$ (child-side leakage).** The child-side aggregation on $u^p_c[t]$ that previously iterated over the variable-length matching set $R_c(t)$ is replaced in Algorithm 2 (line 12) by a single PosJAgg call on (R_p, R_c), whose access trace is determined entirely by $|R_p|$ and $|R_c|$. The discard of unmatched child tuples that previously appeared as an explicit conditional branch is replaced by the merge-sort-and-compact step in Algorithm 3: every tuple of R_{c_j} enters the merged table $\mathcal{M}$ regardless of match status, and the subsequent ObliCompact on the mask $B_{\mathcal{M}}$ touches all $\Lambda + |R_{c_j}|$ entries uniformly. Hence, the trace at every step of child-side processing depends only on $|R_p|$, $|R_c|$, Λ, and $|R_{c_j}|$, all of which are in $\mathcal{L}(Q, \mathcal{T})$.
2) **Closing $\mathcal{L}_2$ (parent-side leakage).** The table $T[key]$ is replaced by the rank-aligned merge of C_p and R_{c_j} in Algorithm 3 (lines 15–16), followed by OAggTree in duplication mode. Both operate on the merged table of

public size $\Lambda + |R_{c_j}|$ and visit every position with a fixed pattern; per-key occupancy never affects the trace.

3) **Closing $\mathcal{L}_3$ (propagation leakage).** The variable-length quantities $|state_p[t]|$ and $\prod_{\ell<j} u^p_{c_\ell}[t]$ enter the access trace only when the naive template iterates over these sets to append intervals or states. We eliminate both by performing all expansion at the root and keeping every subsequent step at a public size. Concretely, Algorithm 2 (line 22) invokes OBLIEXPAND once on (R_{root}, Δ), producing the public-size copy table C of length Λ (the root counts are padded to Λ before this call; Section V-C). From this point onward, propagation generates no new positions; each top-down step in Algorithm 3 only redistributes the existing Λ positions via OBLISORT, OAGGTREE, and OBLICOMPACT on tables of public length $\Lambda + |R_{c_j}|$.
The horizontal-strategy update of $state = (b, \lambda)$ implicitly encodes the left-sibling sum $\sum_{\ell<j} p_{c_\ell}\sigma_{c_\ell}$ through the correction terms $\alpha = idx \bmod \sigma_{c_j}$ and $\beta = (p_{c_j} - rank_{c_j}) \cdot \sigma_{c_j}$. Each copy $x \in C_p$ computes its own (b, λ) by reading only its own fields $(x.pos, x.idx, x.\sigma_{c_j}, x.p_{c_j}, x.rank_{c_j}, x.\lambda)$ at the fixed array index i (Algorithm 3, lines 9 and 27), with no data-dependent address generation, no per-key buffer iteration, and no variable-length append. The number of arithmetic operations is exactly $|C_p| = \Lambda$ regardless of the underlying data, so neither $|state_p[t]|$ nor $\prod_{\ell<j} u^p_{c_\ell}[t]$ is reflected in the trace.

In all three cases, the resulting trace is a deterministic function of $\mathcal{L}(Q, \mathcal{T})$. □

Theorem VI.2 (Obliviousness)**.** *Assume that* POSJAGG, OAGGTREE, OBLIEXPAND, OBLISORT, *and* OBLICOMPACT *are oblivious. Then Algorithms 2 and 3 achieve obliviousness in the sense of Definition II.4 with leakage* $\mathcal{L}(Q, \mathcal{T}) = (\mathcal{T}, |R_1|, \ldots, |R_m|, \Lambda)$.

Proof. By Lemma VI.1, every step that previously induced $\mathcal{L}_1$–$\mathcal{L}_3$ has been replaced by oblivious primitives or fixed-length scans. Each primitive operates on a table whose size is determined by $\mathcal{L}(Q, \mathcal{T})$: $|R_p|+|R_c|$ for POSJAGG, $|R_{root}|$ and Λ for the root OAGGTREE and OBLIEXPAND, and $\Lambda + |R_{c_j}|$ for each top-down sort, duplication, and compaction, since each copy table is maintained at fixed length Λ with dummy records. All remaining operations are element-wise updates or fixed-length scans over these arrays. A PPT simulator $\mathcal{S}$ therefore reproduces the full trace by composing each primitive's simulator on the corresponding public size, yielding a view computationally indistinguishable from the real execution. □

VII. EVALUATION

In this section, we evaluate our experiments on acyclic join workloads. The experiments measure end-to-end latency, parallel scalability, phase-level costs, and the effect of different materialization targets.

TABLE II: Output sizes (rows) of Query 18 and Query 85.

Workload	SF=1	SF=5	SF=10	SF=20	SF=30
Q18	1,380,015	6,890,567	13,781,931	27,564,507	43,149,045
Q85	64,750	329,511	658,083	1,317,422	1,977,620

A. Experimental Settings

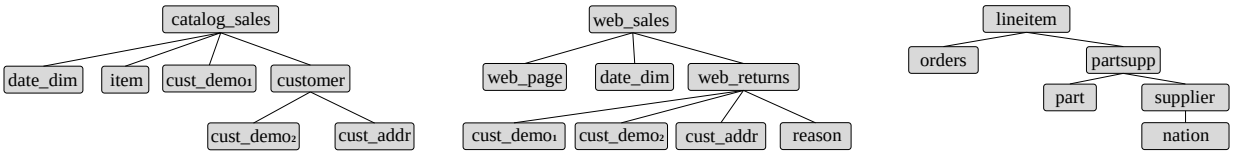
1) Experimental environments: All experiments are conducted inside an enclave of Intel SGXv2. The machine is equipped with an Intel Xeon Platinum 8369B CPU @ 2.70 GHz (16 physical cores, 32 hardware threads with hyperthreading) and a 64 GB enclave page cache (EPC). We pin one worker thread to each physical core and disable hyperthreading, so that the number of parallel workers equals the number of physical cores; consequently the parameter p in our analysis maps directly to physical cores, and the largest configuration we evaluate is $p = 16$. We use this EPC size to avoid EPC paging effects and to focus the comparison on execution-framework cost.

2) Compared Frameworks: We compare three execution frameworks. ObliYan is the state-of-the-art oblivious Yannakakis baseline. ParYan replaces ObliYan's serial oblivious operators with parallel counterparts, but still keeps the Phase III materialization chain. JFYan uses the same parallel oblivious primitives as ParYan, but changes the materialization strategy: it adopts a join-free design that removes this chain and enables branch-level parallelism across sibling subtrees.

3) Workloads: We use acyclic joins derived from TPC-DS [24], [40] and TPC-H, whose join trees are shown in Figure 3. For TPC-DS, we choose Query 18 to test large-output materialization and Query 85 to test a branch-rich join where top-down propagation can exploit sibling parallelism. Table II reports the output cardinalities of the two TPC-DS workloads under all evaluated scale factors. Following the *DO* study [18], we use TPC-H Query 9 for the *DO* experiments so that comparisons under varying privacy parameters (ε, δ) are made on the same prior multi-way join workload. To separate query choice from join-tree shape, Section VII-E also evaluates two Query 85 variants with similar output cardinalities: a chain formed by the rightmost path, and a star formed by the subtree rooted at the web-return relation. For all schemes, we use the same projected inputs by removing attributes that are not needed by the join.

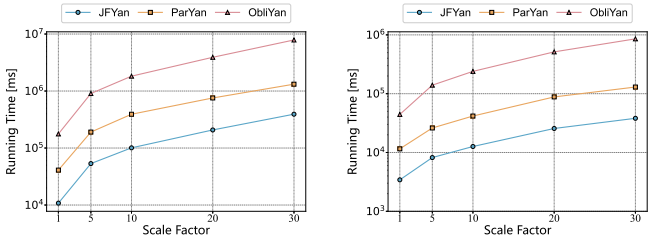
B. Overall Performance and Stage Breakdown

First, we evaluate the overall runtime as the TPC-DS scale factor increases, using $p = 16$ cores, as shown in Figure 4. Both of our designs are compared against the same ObliYan baseline. On Query 18, ParYan achieves a $4.26\times$–$5.97\times$ speedup over ObliYan, while JFYan improves the speedup to $16.37\times$–$20.05\times$. On Query 85, ParYan achieves a $3.85\times$–$6.59\times$ speedup over ObliYan, and JFYan further improves this to $12.96\times$–$22.44\times$. At the largest scale factor, the absolute runtime gap is substantial. For Query 18 at SF=30, JFYan materializes 43,149,045 tuples in 392.40 seconds, while ParYan takes 1,319.08 seconds and ObliYan takes 7,868.75 seconds. For Query 85 at SF=30, JFYan materializes 1,977,620



(a) TPC-DS Query 18 (b) TPC-DS Query 85 (c) TPC-H Query 9

Fig. 3: Join trees of the evaluated workloads.



(a) TPC-DS Query 18 (b) TPC-DS Query 85

Fig. 4: Overall runtime under varying scale factors.

tuples in 38.13 seconds, while ParYan takes 129.75 seconds and ObliYan takes 855.66 seconds. These results separate the effects of the two improvements. ParYan already provides a clear gain over ObliYan by parallelizing the oblivious operators, while JFYan further improves the runtime by removing the Phase III materialization chain. Since the speedups persist on both Query 18, which stresses large-output materialization, and Query 85, which stresses branch-level parallelism, the improvement is not tied to a single favorable query shape.

Second, we break down the runtime by stage to explain the source of the speedup. Figure 5 reports the time spent in Phase I (Up), Phase II (Down), and Phase III (Join) for ObliYan and ParYan, and the corresponding bottom-up contribution computation (Up) and top-down position propagation (Down) stages for JFYan. The breakdown shows that Phase III remains the dominant cost after operator-level parallelization. On Query 18, Phase III accounts for about 81.27%–82.73% of ObliYan's runtime. ParYan reduces the cost of each oblivious operator, but still spends about 91.61%–94.43% of its runtime in Phase III. Since ParYan and JFYan use the same bottom-up computation, we isolate the join-free improvement by comparing JFYan's top-down position propagation with the combined cost of Phase II and Phase III in ParYan. On Query 18, this component speedup is $3.78\times$–$4.45\times$, showing that removing the Phase III materialization chain substantially reduces the cost after the bottom-up phase. Query 85 shows a larger scale-dependent benefit. As the output size increases from $64{,}750$ tuples at SF=1 to $1{,}977{,}620$ tuples at SF=30, the speedup of JFYan's top-down position propagation over ParYan's Phase III materialization increases from $4.32\times$ to $8.35\times$. Over the same range, ParYan's Phase III cost grows from 10.66 seconds to 107.28 seconds, whereas JFYan's top-down propagation cost grows from 2.47 seconds to 12.86 seconds.

C. Primitive-Level Breakdown

In this subsection, we further inspect the primitive-level cost composition. We first evaluate the standalone cost of four basic oblivious primitives under an output-sized workload, where the materialized output length is fixed to 10^6 records and $p = 16$ cores are used. The measured times are 143.69 ms for ObliSort, 6.12 ms for ObliCompact, 9.90 ms for OAggTree, and 210.41 ms for ObliExpand. Thus, ObliExpand is the most expensive primitive.

Figure 5 decomposes the SF=30 runs into the above four oblivious primitives. The fractions are normalized within each algorithm. On Query 18, JFYan's primitive time is distributed mainly among ObliSort (53.80%), ObliExpand (23.16%), and OAggTree (22.15%). In comparison, ParYan spends larger fractions on ObliExpand (33.25%) and OAggTree (27.14%). ObliYan is dominated by ObliSort (78.12%) and ObliExpand (20.94%), reflecting the large repeated sorting and expansion work in output-sized binary materialization. On Query 85, the effect of removing the Phase III materialization chain is more visible. ObliExpand accounts for only 3.73% of JFYan's primitive time, compared with 25.05% for ParYan and 14.18% for ObliYan. Once repeated expansion is removed, ObliSort becomes the main remaining primitive cost. ObliCompact stays below 2% in all six bars, indicating that it has little impact on the observed performance differences. This gap is visible across scale factors and becomes larger as the input size grows.

Figure 6 further compares the time spent in the main oblivious primitives across scale factors. Since these primitives are the core building blocks shared by all three frameworks, this figure focuses on their algorithmic cost differences rather than serving as a full end-to-end runtime. At scale factor 30 on Query 18, ParYan spends $3.51\times$ more ObliExpand time and $3.00\times$ more OAggTree time than JFYan. Compared with JFYan, ObliYan spends $16.66\times$ more ObliExpand time and $26.75\times$ more ObliSort time. Query 85 shows an even larger expansion gap: at scale factor 30, ParYan and ObliYan spend $23.45\times$ and $113.93\times$ more ObliExpand time than JFYan, respectively. These results indicate that JFYan's improvement comes mainly from avoiding repeated output-sized ObliExpand and the sorting work induced by materialized joins.

Figure 7 further localizes the primitive costs by execution stage. For the two Yannakakis-based frameworks, the dominant primitive costs remain in Phase III. On Query 18 at SF=30, ObliYan spends 5.16×10^6 ms on ObliSort and 1.68×10^6 ms on ObliExpand in Phase III. ParYan lowers these costs through operator-level parallelization, but its Phase III still contains large primitive costs: 3.64×10^5 ms for ObliSort, 3.54×10^5 ms for ObliExpand, and 2.85×10^5 ms for OAggTree. We then isolate the join-free improvement by comparing the stages after the shared

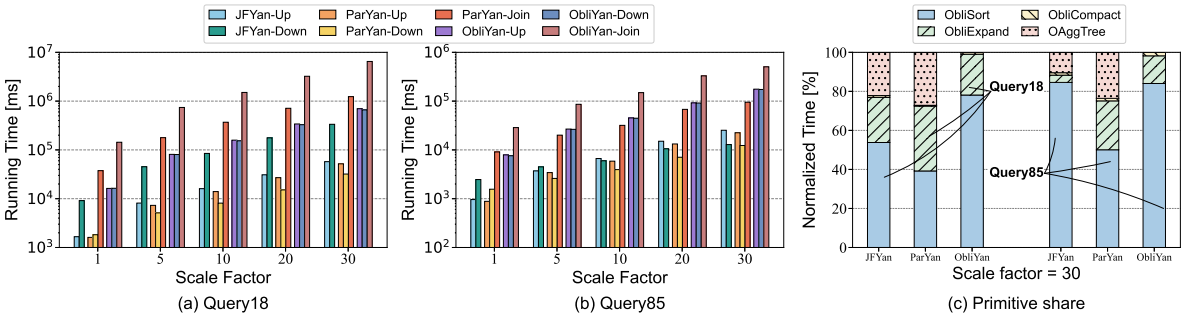


Fig. 5: Stage-level and primitive-level breakdowns.

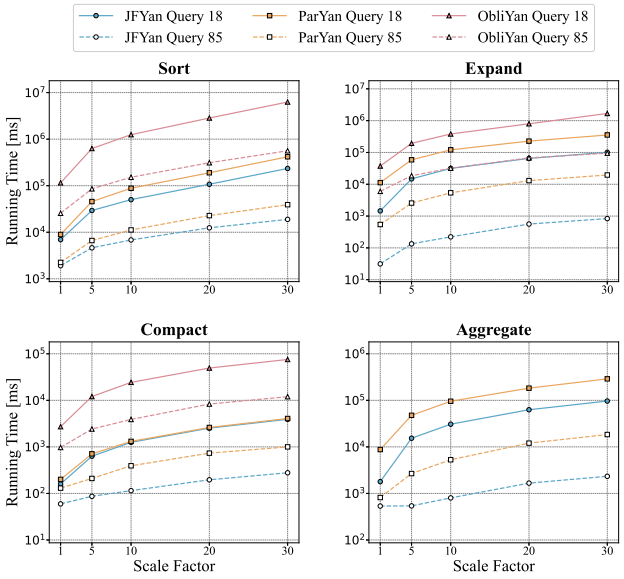


Fig. 6: Primitive-level breakdown under varying scale factors.

bottom-up computation. On Query 18 at SF=30, JFYan's top-down position propagation spends 2.08×10^5 ms on OBLISORT, 1.01×10^5 ms on OBLIEXPAND, and 9.45×10^4 ms on OAGGTREE. The corresponding combined Phase II and Phase III costs in ParYan are 3.91×10^5 ms, 3.54×10^5 ms, and 2.87×10^5 ms, giving reductions of $1.88\times$, $3.51\times$, and $3.04\times$, respectively. On Query 85 at SF=30, the same comparison gives larger reductions of $3.28\times$ for OBLISORT, $23.45\times$ for OBLIEXPAND, and $12.17\times$ for OAGGTREE. Thus, the heatmap is consistent with the stage-level breakdown: the additional gain of JFYan over ParYan comes mainly from replacing the Phase II and Phase III materialization process with top-down position propagation.

D. Parallel Scalability

In this subsection, we evaluate scalability by varying the number of cores p from 1 to 16. Figures 8 and 9 report the speedup of each framework relative to its own single-core execution under the same workload and scale factor. Across the tested scale factors, JFYan gains more from additional cores. At $p = 16$, JFYan achieves a $5.67\times$–$8.09\times$ speedup on Query 18 and a $5.16\times$–$6.00\times$ speedup on Query 85, corresponding to 32%–51% parallel efficiency. In contrast, ParYan reaches $3.66\times$–$4.58\times$ on Query 18 and $3.51\times$–$3.59\times$ on Query 85, corresponding to 22%–29% parallel efficiency. The measured scaling follows the work-span behavior analyzed in Section VI-A. Increasing p reduces the parallel work term, while synchronization between stages and the remaining span determine the part of the runtime that cannot be divided by p. The non-linear efficiency at $p = 16$ is therefore expected: once the work term is reduced, the depth-d propagation span and stage barriers become the visible bottlenecks.

E. Varying Join-Tree Shape

TABLE III: Performance in chain-shape and star-shape tree. All runtimes are reported in milliseconds (ms).

Shape	Metric	SF=1	SF=5	SF=10	SF=20	SF=30
Chain	τ	68,533	343,698	686,586	1,374,361	2,063,370
Star	τ	65,773	329,624	658,320	1,317,909	1,978,354
Chain	Down	1,083.82	2,107.8	2,898.05	4,652.68	5,453.87
Chain	Down+Join	3,528.14	6,188.17	9,017	14,383.9	20,087.8
Chain	Speedup	3.26×	2.94×	3.11×	3.09×	3.68×
Star	Down	1,920.25	3,532.26	4,093.75	5,444.51	6,442.68
Star	Down+Join	6,627.01	12,030.6	16,429.8	26,324.5	35,463.8
Star	Speedup	3.45×	3.41×	4.01×	4.84×	5.50×

In this subsection, we study whether the performance gain depends on the join-tree shape. Figure 10 shows the end-to-end speedup under the two shapes. On the chain-shaped tree, ParYan achieves a $3.72\times$–$5.55\times$ total speedup over ObliYan, while JFYan reaches $10.59\times$–$13.71\times$. On the star-shaped tree, ParYan achieves a $4.08\times$–$5.66\times$ speedup over ObliYan, while JFYan increases from $12.43\times$ at SF=1 to $23.28\times$ at SF=30. Table III further explains the shape sensitivity. We compare JFYan's Down stage with ParYan's combined Down and Join cost to isolate the join-free improvement. In the chain setting, this component speedup is $2.94\times$–$3.68\times$. In the star setting, it increases from $3.45\times$ at SF=1 to $5.50\times$ at SF=30. These results show that JFYan benefits both chain and branch-rich trees, but for different reasons: on chains, the main gain comes from reducing repeated output-size expansion; on star-shaped trees, the gain additionally comes from branch-level parallelism.

F. Comparison across Obliviousness Models

In this subsection, we evaluate whether the speedup of JFYan persists when the public materialization target changes. We use TPC-H Query 9 at scale factor 0.1, whose true output size is $\tau = 600{,}572$. The *a-FO* target is the public AGM

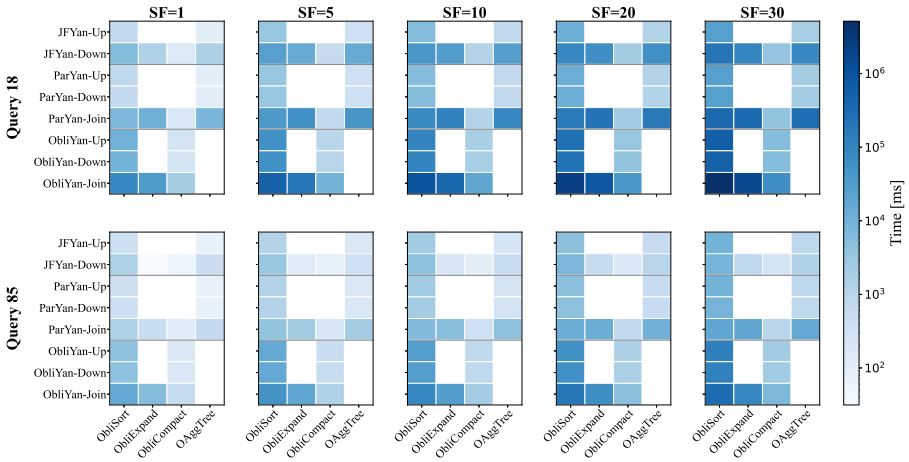
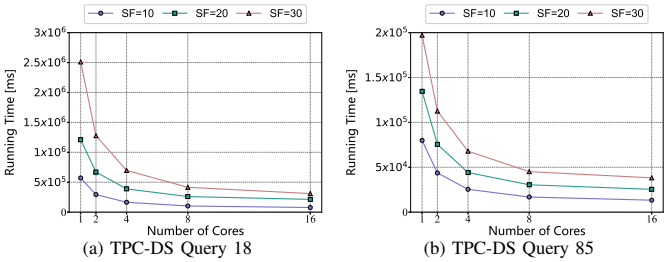
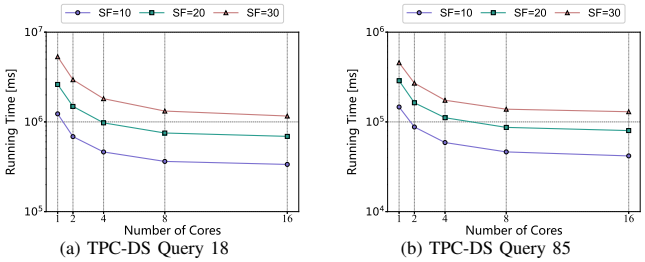


Fig. 7: Per-stage primitive runtime heatmap for TPC-DS Query 18 and Query 85 under varying scale factors. Each panel fixes one scale factor; rows denote algorithm-stage pairs and columns denote oblivious primitives.



(a) TPC-DS Query 18 (b) TPC-DS Query 85

Fig. 8: Core scalability of JFYan.

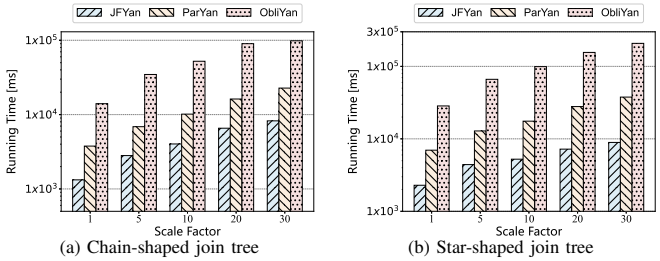


(a) TPC-DS Query 18 (b) TPC-DS Query 85

Fig. 9: Core scalability of ParYan.

bound $\Lambda_{\mathrm{aFO}} = 15{,}014{,}300$. For the *DO* setting, we vary the privacy parameters and use the resulting differentially private target Λ_{DO}. Λ_{DO} is computed following Wu et al. [18], and the reported runtimes include this target-computation cost.

Table IV shows that JFYan's advantage is preserved under different public targets. Across all *DO* rows, JFYan is $9.6\times$–$10.6\times$ faster than ObliYan and $2.8\times$–$3.7\times$ faster than ParYan.



(a) Chain-shaped join tree (b) Star-shaped join tree

Fig. 10: Overall runtime under varying join-tree shape.

The speedup remains stable because changing the public target affects all frameworks, but it does not change their materialization structure. The effect of the public target is also clear from the table. A stronger *DO* setting leads to a larger Λ_{DO} and therefore increases the runtime of all schemes. The final row further shows the *a-FO* setting with the AGM target $\Lambda_{\mathrm{aFO}} = 15{,}014{,}300$, where JFYan still achieves a $3.4\times$ speedup over ParYan and a $9.7\times$ speedup over ObliYan. This confirms that the same target-adaptive implementation remains effective across both *DO* and *a-FO*.

TABLE IV: Performance in different public padding targets. All runtimes are reported in milliseconds (ms).

Model	ε	δ	Target Λ	JFYan	ParYan	ObliYan [17]
DO	8	10^{-8}	2,234,525	15,208.5	43,099.3	155,038
DO	4	10^{-8}	3,388,454	17,820.1	60,562.5	171,393
DO	2	10^{-8}	5,766,052	34,772.2	106,238	342,609
DO	1.8	10^{-8}	9,551,183	70,785.7	200,282	748,991
DO	1.5	10^{-8}	24,271,011	158,784	501,265	1,666,830
DO	4	10^{-8}	3,388,454	17,820.1	60,562.5	171,393
DO	4	10^{-10}	3,924,104	19,586.3	71,990.3	188,422
DO	4	10^{-12}	4,459,754	31,939.7	90,046.3	329,545
DO	4	10^{-14}	4,995,403	32,953.9	98,236.3	334,470
DO	4	10^{-16}	5,531,053	34,522.9	111,119	341,551
a-FO	–	–	15,014,300	81,842.8	280,147.59	793,035

G. Evaluation of Workloads, Obliviousness Overhead, and Memory Use

In this subsection, we further examine how join-tree structure, data scale, oblivious execution, and EPC capacity affect JFYan.

1) Different Workload Structures and Real-World Workloads: The structural workloads are derived from *email-EuAll*, a directed email network from the Stanford Large Network Dataset Collection [41]. Each join-tree node is an alias of `Edge(source,target)`, and each parent–child edge applies the predicate `parent.target=child.source`. We construct three query families by varying the join-tree depth $d \in \{2,3,4\}$, maximum branching factor $k \in \{2,3,4\}$, and

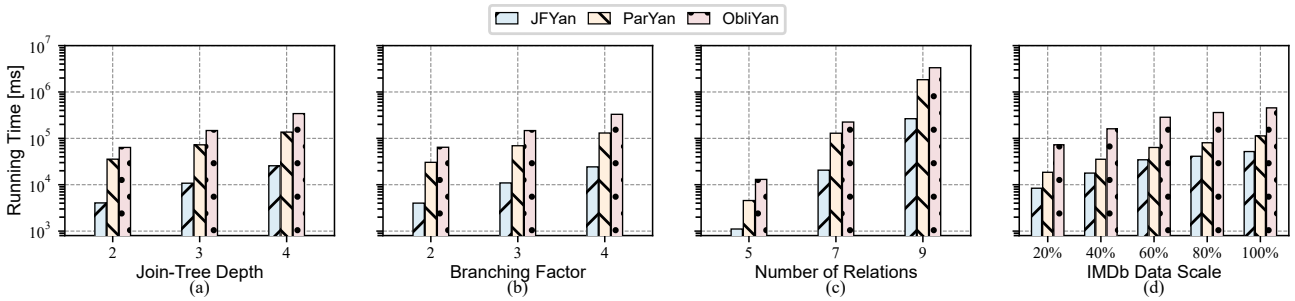
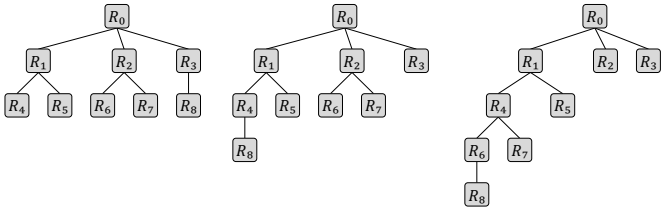


Fig. 11: Runtime on email-EuAll and JOB 13d.

number of relations $m \in \{5, 7, 9\}$, respectively. Figures 12–14 show the resulting trees and their input and output cardinalities. We further evaluate the nine-relation acyclic core of JOB 13d over IMDb [42] in Figure 15. To vary its data size, we hash `movie_id` values into 100 stable buckets and retain the first 20, 40, 60, 80, or 100 buckets. Table V reports the resulting input and output cardinalities. Together, these workloads cover variations in the number of relations, join-tree depth, branching factor, and output-to-input ratio, as well as the natural data skew of two real datasets.

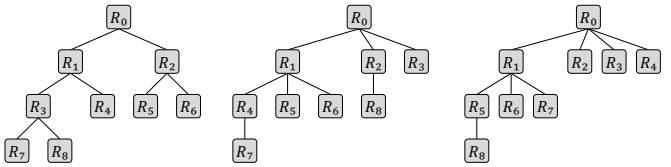


(a) Depth = 2
Input = 26,586; Output = 756,224

(b) Depth = 3
Input = 29,540; Output = 1,701,504

(c) Depth = 4
Input = 32,494; Output = 3,828,384

Fig. 12: Varying join-tree depth.

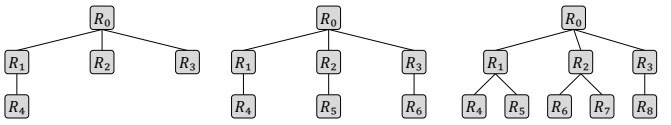


(a) Branching factor = 2
Input = 26,136; Output = 743,424

(b) Branching factor = 3
Input = 29,040; Output = 1,672,704

(c) Branching factor = 4
Input = 31,944; Output = 3,763,584

Fig. 13: Varying branching factor.



(a) 5 relations
Input = 21,780; Output = 352,836

(b) 7 relations
Input = 30,492; Output = 3,175,524

(c) 9 relations
Input = 39,204; Output = 28,579,716

Fig. 14: Varying the number of relations.

We run these workloads with $p = 16$ physical cores under the same experimental settings as Section VII-A. Figure 11 reports end-to-end runtime. On the email-EuAll workloads, which contain 21,780–39,204 input rows and produce 352,836–28,579,716 output rows, JFYan is $4.1\times$–$8.8\times$ faster than ParYan and $10.9\times$–$16.0\times$ faster than ObliYan. It is the fastest method at every tested depth, branching factor, and relation count. On JOB 13d, the input grows from 4,358,244 to 21,438,043 rows and the output from 133,293 to 670,390 rows; JFYan is $1.8\times$–$2.2\times$ faster than ParYan and $8.3\times$–$9.0\times$ faster than ObliYan. JOB 13d complements email-EuAll with the natural skew of IMDb data and a much lower output-to-input ratio. JFYan therefore remains the fastest method across widely different output-to-input ratios.

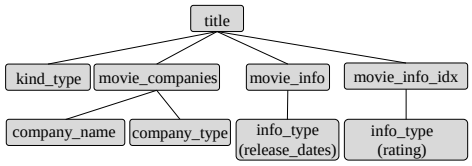


Fig. 15: Join tree of the JOB 13d acyclic core.

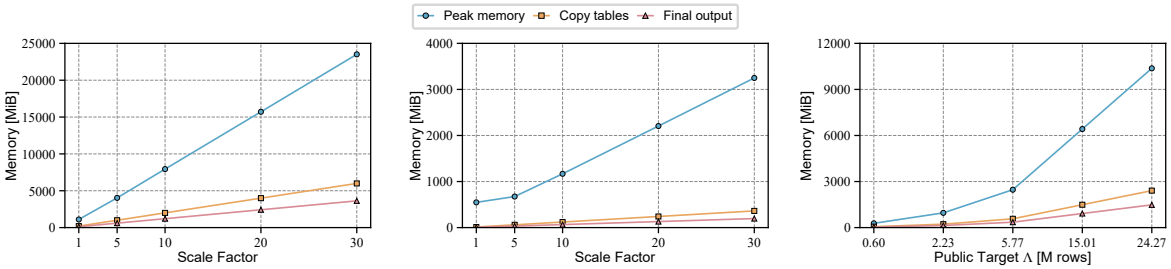
TABLE V: JOB 13d data scales.

Data scale	20%	40%	60%	80%	100%
Input (rows)	4,358,244	8,634,195	12,896,390	17,171,737	21,438,043
Output (rows)	133,293	269,135	399,337	537,807	670,390

2) Obliviousness Overhead: We compare JFYan with a non-oblivious implementation, NonObliJFYan, to measure the overhead of oblivious execution. NonObliJFYan retains the same bottom-up contribution computation and top-down position assignment, but replaces oblivious primitives with hash lookups, data-dependent branches, and direct writes. Both implementations evaluate the same query and produce identical output; their runtime difference therefore measures the cost of hiding memory-access patterns.

Figure 17 compares their runtime over scale factors 1–30 with $p = 16$ physical cores. Relative to NonObliJFYan, JFYan's runtime overhead increases from $13.8\times$ to $53.2\times$ on Query 18 and from $5.7\times$ to $24.3\times$ on Query 85. At SF=30, Query 18 takes 392,398 ms with JFYan and 7,377 ms with NonObliJFYan; the corresponding Query 85 runtimes are 38,133 ms and 1,568 ms. The gap widens because JFYan sorts and processes fixed-size arrays independently of data values, whereas NonObliJFYan uses data-dependent hash accesses and direct writes.

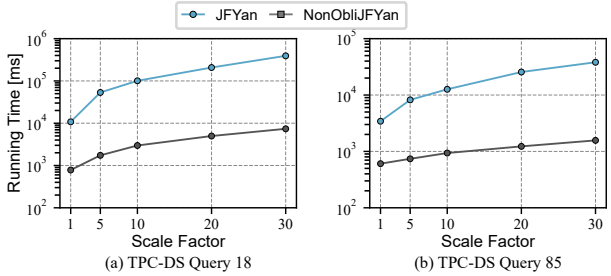
3) Memory Consumption: JFYan uses additional copy-table storage to avoid repeatedly materializing large intermediate joins. With $p = 16$ physical cores, we measure its peak memory, copy-table storage, and final-output storage under different data scales and public output targets. Figure 16 reports the results. At SF=30, Query 18 reaches a peak of 23.51 GB, including 5.99 GB of copy tables and 3.63 GB of



(a) TPC-DS Query 18 (b) TPC-DS Query 85 (c) TPC-H Query 9

Fig. 16: Memory consumption of JFYan.

final output; Query 85 reaches 3.25 GB, including 0.36 GB of copy tables and 0.20 GB of final output. On the same workloads, JFYan is 20.05× and 22.44× faster than ObliYan, respectively. For TPC-H Query 9, increasing the public target from the true output size $\tau = 600{,}572$ to $\Lambda_{\mathrm{DO}} = 24{,}271{,}011$ raises peak memory from 0.28 to 10.37 GB. Thus, the additional memory grows roughly with the public output size and remains moderate relative to the 64 GB EPC used in our main experiments. This trade-off is particularly suitable for modern SGX platforms, where growing EPC capacities place greater emphasis on execution efficiency.



(a) TPC-DS Query 18 (b) TPC-DS Query 85

Fig. 17: Runtime with and without oblivious execution.

TABLE VI: TPC-DS Query 18 runtime with a 32 GB EPC (ms).

SF	Output (rows)	JFYan	ParYan	ObliYan
30	43,149,045	866,083	3,141,730	12,814,100
50	68,882,722	1,662,810	6,474,990	30,778,100

4) Performance under EPC Paging: We conduct three experiments to evaluate JFYan under increasing EPC pressure. First, we reduce the EPC from 64 GB to 32 GB while keeping Query 18 at SF=30 and $p = 16$. As reported in Table VI, JFYan takes 866,083 ms, compared with 12,814,100 ms for ObliYan, and remains 14.80× faster. Second, we increase the scale factor to 50, for which Query 18 produces 68,882,722 rows. All three methods incur EPC paging; for ObliYan, `ksgxd` accumulates 1,540.16 s of CPU time. JFYan completes in 1,662,810 ms and is 18.51× faster than ObliYan. Third, we increase the scale factor to 100, where Query 18 produces 137,746,996 rows. JFYan reaches 52.90 GB of peak memory, exceeding the 32 GB EPC. During the run, `ksgxd` accumulates 1,437.41 s of CPU time, and swap usage increases by as much as 20.71 GB. Despite this sustained paging, JFYan completes in 5,417,180 ms, which is still 5.68× faster than ObliYan at SF=50 although JFYan processes twice the scale factor. These results show that JFYan remains effective as EPC pressure increases.

VIII. Related Work

We review related work on oblivious query processing and oblivious joins.

Oblivious query processing. Arasu and Kaushik [12] first studied oblivious query processing for encrypted databases and proposed oblivious algorithms for selections, joins, grouping, and aggregation. ObliDB [19] later built an enclave-based system with oblivious relational operators, showing the practicality of access pattern protection in encrypted query processing. Since worst-case padding in oblivious query processing can be expensive, ADORE [43] introduced differential obliviousness for relational operators, and Doquet [44] further supported *DO* query processing with private data structures and indices under a stronger threat model. Recent work has also identified oblivious graph query processing as an open problem [14].

Oblivious joins. A line of work studies oblivious binary join operators. Krastnikov et al. [13] proposed sorting-based oblivious equi-joins. Chang et al. [21], [22] developed ORAM-based techniques for equi-joins and band joins, while Mavrogiannakis et al. [15] designed parallel oblivious binary joins in shared memory. Distributed oblivious join processing has also been explored in Opaque [31], SODA [32], and JODES [33]. However, these works focus on a single binary join operator. For multi-way joins, Chang et al. [21], [22] extended ORAM-based index techniques to acyclic equi-joins, but ORAM incurs high overhead. Hu and Wu [17] proposed sorting-based oblivious algorithms without ORAM and achieved worst-case-optimal complexity up to logarithmic factors for acyclic and cyclic joins. Wu et al. [18] further introduced differentially oblivious multi-way joins to avoid padding to worst-case output sizes. Nevertheless, existing oblivious multi-way join algorithms still follow a bottom-up materialization strategy and construct outputs through intermediate relations. Our work removes this materialization chain for acyclic joins by using one root expansion and top-down position propagation, enabling branch-level parallelism across the join tree.

IX. Conclusion

We presented an oblivious algorithm for acyclic joins that eliminates the Yannakakis join materialization phase, replacing it with bottom-up contribution computation and top-down position propagation. The algorithm reduces the number of ObliExpand invocations from $\Theta(m)$ in the state-of-the-art baseline ObliYan to exactly one, and admits branch-level parallelism that the materialization phase did not. On p cores it runs in $O\big((kN+m\Lambda)\log^2(\Lambda+N)/p+d\log^2(\Lambda+N)\big)$, where the single public target Λ instantiates *FO*, *a-FO*, or *DO*. We complement the analysis with an empirical study on standard acyclic workloads.

Several directions remain open for future work. First, extending our framework to cyclic joins is a natural next step, where the absence of a join tree makes position propagation more complex. Second, adapting the algorithm to distributed settings, where intermediate results must be exchanged across nodes without leaking their sizes, presents both algorithmic and systems challenges worth exploring.

References

[1] H. Hacigumus, B. Iyer, and S. Mehrotra, "Providing database as a service," in *Proceedings 18th International Conference on Data Engineering (ICDE)*, 2002, pp. 29–38.

[2] Y. Yang, D. Papadias, S. Papadopoulos, and P. Kalnis, "Authenticated join processing in outsourced databases," in *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 2009, pp. 5–18.

[3] X. Chen, J. Li, J. Ma, Q. Tang, and W. Lou, "New algorithms for secure outsourcing of modular exponentiations," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 25, no. 9, pp. 2386–2396, 2013.

[4] R. A. Popa, C. M. Redfield, N. Zeldovich, and H. Balakrishnan, "Cryptdb: processing queries on an encrypted database," *Communications of the ACM*, vol. 55, no. 9, pp. 103–111, 2012.

[5] F. Hahn, N. Loza, and F. Kerschbaum, "Joins over encrypted data with fine granular security," in *IEEE 35th international conference on data engineering (ICDE)*, 2019, pp. 674–685.

[6] C. Jutla and S. Patranabis, "Efficient searchable symmetric encryption for join queries," in *International conference on the theory and application of cryptology and information security (ASIACRYPT)*, 2022, pp. 304–333.

[7] M. Shafieinejad, S. Gupta, J. Y. Liu, K. Karabina, and F. Kerschbaum, "Equi-joins over encrypted data for series of queries," in *IEEE 38th international conference on data engineering (ICDE)*, 2022, pp. 1635–1648.

[8] K. Maliszewski, J.-A. Quiané-Ruiz, J. Traub, and V. Markl, "What is the price for joining securely?" *Proceedings of the VLDB Endowment (VLDB)*, vol. 15, no. 3, pp. 659–672, 2021.

[9] K. Maliszewski, J.-A. Quiané-Ruiz, and V. Markl, "Cracking-like join for trusted execution environments," *Proceedings of the VLDB Endowment (VLDB)*, vol. 16, no. 9, pp. 2330–2343, 2023.

[10] M. Li, X. Zhao, L. Chen, C. Tan, H. Li, S. Wang, Z. Mi, Y. Xia, F. Li, and H. Chen, "Encrypted databases made secure yet maintainable," in *17th USENIX symposium on operating systems design and implementation (OSDI 23)*, 2023, pp. 117–133.

[11] S. Cao, Z. Niu, and G. Li, "Letindex: A secure learned index with tee," *Proceedings of the VLDB Endowment (VLDB)*, vol. 18, no. 12, pp. 5403–5406, 2025.

[12] A. Arasu and R. Kaushik, "Oblivious query processing," in *Proceedings of the International Conference on Database Theory (ICDT)*, 2014, pp. 26–37.

[13] S. Krastnikov, F. Kerschbaum, and D. Stebila, "Efficient oblivious database joins," *Proceedings of the VLDB Endowment (VLDB)*, vol. 13, no. 11, 2020.

[14] B. Bhattacharjee, N. Ahmed, R. Miller, and S. Maiyya, "Towards oblivious property graph databases," in *Proceedings of the 8th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, 2025, pp. 1–6.

[15] A. Mavrogiannakis, X. Wang, I. Demertzis, D. Papadopoulos, and M. Garofalakis, "Obliviator: oblivious parallel joins and other operators in shared memory environments," in *Proceedings of the 34th USENIX Conference on Security Symposium (USENIX Security*, 2025, pp. 8521–8540.

[16] C. Sahin, A. Magat, V. Zakhary, A. El Abbadi, H. Lin, and S. Tessaro, "Understanding the security challenges of oblivious cloud storage with asynchronous accesses," in *IEEE 33rd International Conference on Data Engineering (ICDE)*, 2017, pp. 1377–1378.

[17] X. Hu and Z. Wu, "Optimal oblivious algorithms for multi-way joins," in *28th International Conference on Database Theory (ICDT)*, 2025, pp. 25:1–25:19.

[18] Z. Wu, W. Dong, and X. Hu, "Differentially oblivious multi-way join," *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, vol. 4, no. 3, pp. 1–26, 2026.

[19] S. Eskandarian and M. Zaharia, "Oblidb: Oblivious query processing for secure databases," *Proceedings of the VLDB Endowment (VLDB)*, vol. 13, no. 2, 2019.

[20] I. Demertzis, "Large-scale private computation for real-world applications via trusted hardware and obliviousness," in *IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 4483–4485.

[21] Z. Chang, D. Xie, S. Wang, and F. Li, "Towards practical oblivious join," in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 2022, pp. 803–817.

[22] Z. Chang, D. Xie, S. Wang, F. Li, and Y. Shen, "Towards practical oblivious join processing," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 36, no. 4, pp. 1829–1842, 2024.

[23] M. Yannakakis, "Algorithms for acyclic database schemes," in *Proceedings of the VLDB Endowment (VLDB)*, vol. 81, 1981, pp. 82–94.

[24] A. Pagh and R. Pagh, "Scalable computation of acyclic joins," in *Proceedings of the twenty-fifth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems (PODS)*, 2006, pp. 225–232.

[25] M. Idris, M. Ugarte, and S. Vansummeren, "The dynamic yannakakis algorithm: Compact and efficient query processing under updates," in *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, 2017, pp. 1259–1274.

[26] Y. Wang and K. Yi, "Secure yannakakis: Join-aggregate queries over private data," in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 1969–1981.

[27] N. Tziavelis, W. Gatterbauer, and M. Riedewald, "Toward responsive dbms: optimal join algorithms, enumeration, factorization, ranking, and dynamic programming," in *IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 3205–3208.

[28] H. Q. Ngo, C. Ré, and A. Rudra, "Skew strikes back: new developments in the theory of join algorithms," *ACM SIGMOD Record*, vol. 42, no. 4, pp. 5–16, 2014.

[29] Q. Wang, B. Chen, B. Dai, K. Yi, F. Li, and L. Lin, "Yannakakis+: Practical acyclic query evaluation with theoretical guarantees," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 3, no. 3, pp. 1–28, 2025.

[30] P. A. Bernstein and D.-M. W. Chiu, "Using semi-joins to solve relational queries," *Journal of the ACM (JACM)*, vol. 28, no. 1, pp. 25–40, 1981.

[31] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017, pp. 283–298.

[32] X. Li, N. Sun, Y. Luo, and M. Gao, "Soda: A set of fast oblivious algorithms in distributed secure data analytics," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1671–1684, 2023.

[33] Y. Wang, X. Zeng, S. Wang, and F. Li, "Jodes: Efficient oblivious join in the distributed setting," *Proceedings of the VLDB Endowment (VLDB)*, vol. 18, no. 5, pp. 1291–1304, 2025.

[34] J. G. Chamani, I. Demertzis, D. Papadopoulos, C. Papamanthou, and R. Jalili, "Graphos: Towards oblivious graph processing," *Proceedings of the VLDB Endowment (VLDB)*, vol. 16, no. 13, pp. 4324–4338, 2023.

[35] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the intel {SGX} kingdom with transient {Out-of-Order} execution," in *27th USENIX Security Symposium (USENIX Security)*, 2018, pp. 991–1008.

[36] G. Blelloch, D. Ferizovic, and Y. Sun, "Joinable parallel balanced binary trees," *ACM Transactions on Parallel Computing*, vol. 9, no. 2, pp. 1–41, 2022.

[37] M. F. Ionescu and K. E. Schauser, "Optimizing parallel bitonic sort," in *Proceedings of the IEEE International Parallel Processing Symposium (IPPS)*, 1997, pp. 303–309.

[38] N. Ngai, I. Demertzis, J. G. Chamani, and D. Papadopoulos, "Distributed & scalable oblivious sorting and shuffling," in *IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 4277–4295.

[39] S. Sasy, A. Johnson, and I. Goldberg, "Fast fully oblivious compaction and shuffling," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2565–2579.

[40] M. Poess, R. O. Nambiar, and D. Walrath, "Why you should run tpc-ds: A workload analysis," in *Proceedings of the VLDB Endowment (VLDB)*, vol. 7, 2007, pp. 1138–1149.

[41] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," https://snap.stanford.edu/data/, 2014, accessed August 2026.

[42] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[43] L. Qin, R. Jayaram, E. Shi, Z. Song, D. Zhuo, and S. Chu, "Adore: Differentially oblivious relational database operators," *Proceedings of the VLDB Endowment (VLDB)*, vol. 16, no. 4, pp. 842–855, 2022.

[44] L. Qiu, G. Kellaris, N. Mamoulis, K. Nissim, and G. Kollios, "Doquet: Differentially oblivious range and join queries with private data structures," *Proceedings of the VLDB Endowment (VLDB)*, vol. 16, no. 13, pp. 4160–4173, 2023.